

ADVERSARIAL ATTACKS LEVERAGE INTERFERENCE BETWEEN FEATURES IN SUPERPOSITION

Anonymous authors
Anonymous submission

ABSTRACT

Fundamental questions remain concerning the conditions under which adversarial examples (AExs) arise in neural networks (NNs) and the mechanisms by which small input perturbations alter model predictions. Existing accounts typically characterise these examples either as consequences of irregular decision-boundary geometry or as responses to predictive but non-robust input features. In this paper, we argue instead that adversarial vulnerability can arise from *efficient* representational encoding in NNs. Specifically, we analyse how superposition – the representation of more features than available dimensions – induces latent feature configurations that adversarial procedures can exploit. We show that adversarial perturbations leverage interference among superposed features, so that attack directions become predictable from the geometry of the learned feature representations. This framework provides a mechanistic explanation for two established phenomena: adversarial attack transferability between models trained under similar regimes and class-specific patterns of vulnerability. In synthetic settings with precisely controlled superposition, we establish that superposition *suffices* to induce adversarial vulnerability. We then show that the same qualitative mechanisms are observable in a vision transformer (ViT) trained on CIFAR-10. Together, these results indicate that adversarial vulnerability can be a byproduct of representational compression, rather than solely a defect of the learning procedure or an intrinsic property of non-robust inputs. We release our implementation through a software repository.

1 INTRODUCTION

Despite extensive work on AExs (Szegedy et al., 2014; Goodfellow et al., 2014), their underlying causal mechanism remains unresolved. In particular, existing theory does not yet provide a complete account of how small perturbations in input space can induce large changes in model predictions, which perturbation directions should succeed, why attacks transfer between independently trained models, or how defences should be specified from first principles.

Existing explanations can be grouped into two broad classes (Nakkiran, 2019). The *bug* perspective attributes vulnerability to properties such as deficiencies in the learning process (Schmidt et al., 2018; Nakkiran, 2019) or the geometry of high-dimensional decision boundaries (Fawzi et al., 2016). Analyses in this line often provide statistical bounds on capacity (Bubeck & Sellke, 2021) or Lipschitz behaviour (Hein & Andriushchenko, 2017), but they typically do not identify the particular perturbation components that succeed on a given representation. The *feature* perspective instead proposes that AExs exploit predictive but non-robust statistical regularities in the data (Ilyas et al., 2019), thereby shifting the explanatory focus to the relation between data features and human perceptual categories. In this view, non-robust features are usually treated as fixed properties of the data distribution, rather than as quantities whose effect may depend on the network’s internal encoding strategy. Consequently, neither perspective fully specifies how architectural representational constraints interact with semantic structure in the input distribution.

In this paper, we connect these accounts through a mechanistic interpretability analysis of the interaction between architectural constraints and data semantics (*i.e.* human interpretable properties of the data). We show that AExs can arise from interference among learned representations in NNs. Adversarial optimisation systematically exploits superposed latent structure – a representational mechanism that increases effective capacity – to construct perturbations that control model outputs. The resulting causal pathway is explicit: input correlations constrain latent feature arrangements, those arrangements determine interference coefficients, and the interference coefficients specify

attack directions and transferability. This framework therefore supports predictions about which perturbations succeed and why the same perturbations transfer across models.

Our account builds on two hypotheses about neural information processing: the linear representation hypothesis (LRH) (Park et al., 2024) and the theory of superposition (Elhage et al., 2022). The LRH states that *input features*—abstract variables of the data—are encoded as linear directions in a network representation space. Superposition further hypothesises that NNs can encode many more such features than they have individual neurons or activation dimensions by assigning multiple features to shared, non-orthogonal directions. This allocation increases representational efficiency but introduces interference, so that perturbing one represented feature can produce systematic changes in other represented features. We study whether this interference constitutes an exploitable vulnerability mechanism for AExs, and what it implies for attack transfer and class-specific susceptibility. Our primary contributions are:

- We formulate a mechanism that links architectural representational constraints to feature-level properties in adversarial vulnerability. In simple models, we show that data statistics induce specific superposition geometries and that these geometries determine the realised adversarial perturbations. The resulting interference patterns explain attack transferability and class-specific susceptibility.
- We show that the same qualitative behaviour persists in a ViT trained on CIFAR-10 after inserting an engineered bottleneck layer that induces class-feature superposition.
- We establish that superposition is sufficient, but not necessary, for adversarial vulnerability. This distinction identifies algorithmic brittleness as a separate mechanism of attack susceptibility.
- We show that mechanistic knowledge of learned representations can be used to construct informed adversarial attacks, supporting the case for defences that target the relevant represented semantics.

2 BACKGROUND

We first specify the representational assumptions used throughout the analysis: the LRH and the superposition hypothesis. Let $\mathbf{x} \in \mathcal{X}$ denote an input to a NN, and let $\mathbf{h}^{(l)} \in \mathbb{R}^{d_l}$ denote the activation vector at layer l .

Linear representation hypothesis (LRH). The LRH posits that NNs encode many computational variables, including semantic properties of the input, as linear directions in activation space that can be manipulated as representational abstractions (Park et al., 2024; Guerner et al., 2023). This linear encoding bias is hypothesised to be useful because linear separability makes features readily detectable, and because subsequent layers can process directional features efficiently through dot products with learned weights. Empirical evidence for this hypothesis is increasing (Gurnee et al., 2023; Park et al., 2024). Let $\mathcal{C} = \{c_1, \dots, c_M\}$ denote a collection of M semantically meaningful latent features (e.g. concepts like “presence of shape,” or “indoor vs. outdoor”). Formally:

Definition 1 (Linear Representation Hypothesis (LRH)) A neural network layer with activations $\mathbf{h}^{(l)} \in \mathbb{R}^{d_l}$ satisfies the LRH if it represents the latent features $\mathcal{C} = \{c_1, \dots, c_M\}$ as linear directions $\{\mathbf{v}_j\}_{j=1}^M \subset \mathbb{R}^{d_l}$ such that:

$$\mathbf{h}^{(l)}(\mathbf{x}) \approx \sum_{j=1}^M a_j(\mathbf{x}) \mathbf{v}_j$$

where $a_j(\mathbf{x}) \in \mathbb{R}$ represents the activation magnitude of feature c_j , and $\mathbf{v}_j$ is the corresponding linear direction. The features are **linearly accessible** (Costa et al., 2025): inputs $\mathbf{x}_0, \mathbf{x}_1 \in \mathcal{X}$ that differ mainly in the value of feature c_j while holding other features approximately constant (i.e., $a_j(\mathbf{x}_1) - a_j(\mathbf{x}_0) = k$ and $|a_i(\mathbf{x}_1) - a_i(\mathbf{x}_0)| < \lambda$ for all $i \neq j$ and some small $\lambda > 0$) satisfy:

$$\mathbf{h}^{(l)}(\mathbf{x}_1) - \mathbf{h}^{(l)}(\mathbf{x}_0) \approx k \mathbf{v}_j$$

where k reflects the change in c_j .

Superposition and sparsity. NNs may encode more latent features than the dimensionality of the relevant activation space permits under an orthogonal basis: $M > d_l$. For example, LLMs can refer to far more place names than there are residual-stream dimensions. Superposition provides one formal account of this regime: the network represents features using an overcomplete, non-orthogonal set of directions $\{\mathbf{v}_j\}_{j=1}^M$. This construction exploits the fact that $2^{\Theta(d\epsilon^2)}$ nearly orthogonal

vectors ($< \epsilon$ cosine similarity) can be embedded in d dimensions (Tkocz, 2012), together with the observation that sparse vectors can often be recovered accurately after projection into lower-dimensional spaces (Elhage et al., 2022; Bereska & Gavves, 2024; Sawmya et al., 2025). This representational regime introduces two technical complications. First, *polysemanticity* (Scherlis et al., 2022; Lecomte et al., 2023) occurs when a single neuron participates in multiple feature directions, so the scalar activation of that neuron no longer corresponds to one interpretable concept. Second, non-orthogonal feature directions induce *interference*: activation along one feature direction changes the readout of other feature directions. Networks can partially control this interference using nonlinear operations (e.g. ReLU, softmax) that disambiguate superposed features (Gurnee et al., 2023).

Definition 2 (Superposition Hypothesis) *A network layer represents features in superposition if:*

1. *The number of latent features exceeds layer dimensionality: $M > d_l$*
2. *There exists non-orthogonal feature directions: $\exists i, j$ with $i \neq j$ such that $\mathbf{v}_i \cdot \mathbf{v}_j \neq 0$*
3. *Latent feature activations are sparse: $\mathbb{E}_{\mathbf{x}}[\|\mathbf{a}(\mathbf{x})\|_0] \ll M$, where $\mathbf{a}(\mathbf{x}) = [a_1(\mathbf{x}), \dots, a_M(\mathbf{x})]^T$*

The network trades representational capacity against feature interference by packing more features than dimensions.

Adversarial attacks. We use the following standard formulation of adversarial perturbations and attacks (Szegedy et al., 2014; Costa et al., 2024):

Definition 3 (Adversarial attack) *Let $\mathbb{B}_p(\mathbf{x}, \epsilon)$ denote the ϵ -ball around $\mathbf{x} \in \mathcal{X}$ in the ℓ_p -sense, e.g. $\mathbb{B}_\infty$ is the ℓ_∞ -ball. Any small δ that p -perturbs $\mathbf{x}$ to create an **adversarial input** $\mathbf{x}' \in \mathbb{B}_p(\mathbf{x}, \epsilon)$ such that $f(\mathbf{x}') \neq f(\mathbf{x})$ (untargeted) or $f(\mathbf{x}') = y_{\text{target}}$ (targeted) is known to be an **adversarial attack**. ℓ_∞ - or ℓ_2 -norm attacks are created via the projected gradient descent (PGD) (detailed in Sec. A).*

Definition 4 (Adversarial robustness) *A classifier $g_w(\cdot)$ with parameters w is said to be **robust** at $\mathbf{x}$ with radius $\epsilon > 0$, if $g_w(\mathbf{x}') = g_w(\mathbf{x}) \forall \mathbf{x}' \in \mathbb{B}(\mathbf{x}, \epsilon)$.*

3 SUPERPOSITION GEOMETRY CAN DETERMINE ADVERSARIAL ATTACKS

We begin by testing whether adversarial attacks exploit interference induced by superposed latent features, and by characterising the geometric conditions under which such exploitation occurs. Specifically, we evaluate three questions:

1. *Do adversarial perturbations exploit the interference between superposed features?*
2. *Do correlations in the input shape the geometric arrangement of superposed latent features?*
3. *Can shared latent geometry explain why attacks transfer between independently trained models?*

Experimental setup. To address these questions, we construct a synthetic classification task that instantiates established LRH and superposition phenomena (Park et al., 2024; Elhage et al., 2022) under controlled experimental conditions. The task is specified to: (1) provide a transparent classification problem for analysing AExs; (2) encode class concepts as linear directions, as assumed by the LRH; (3) impose controllable superposition among those latent features; and (4) preserve known input-to-feature correspondences, allowing direct tests of mechanistic predictions about adversarial perturbations.

We partition each input $\mathbf{x} \in \mathbb{R}^d$ into k equal-sized groups, $\mathbf{x} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k)}]$, where $\mathbf{x}^{(j)}$ contains the p input features associated with class j . The label is determined by the group with maximal total activation: $y = \operatorname{argmax}_{j \in \{1, \dots, k\}} \sum_{i=1}^p x_i^{(j)}$. Thus, each $x_i^{(j)}$ functions as evidence for class j , and the p features correspond to distinct class attributes. Following Elhage et al. (2022), we sample $x_i^{(j)} \sim \text{Uniform}(0, 1)$ with sparsity S (probability of setting to zero).

A two-layer network maps the input into an m -dimensional bottleneck representation, $\mathbf{h} = \text{ReLU}(\mathbf{W}_e \mathbf{x} + \mathbf{b}_e) \in \mathbb{R}^m$ with $m < k < d$, and then decodes logits by $\mathbf{z} = \mathbf{W}_d \mathbf{h} + \mathbf{b}_d \in \mathbb{R}^k$. Our primary configuration uses cross entropy (CE) loss without ReLUs or biases, chosen because it isolates the linear representational geometry while the softmax limits interference by exponentially suppressing contributions from non-target latent features. To verify that the conclusions are not an artefact of this simplified configuration, we also evaluate a second setting using mean squared error (MSE) loss with ReLUs and biases. In all cases, the network must compress information from

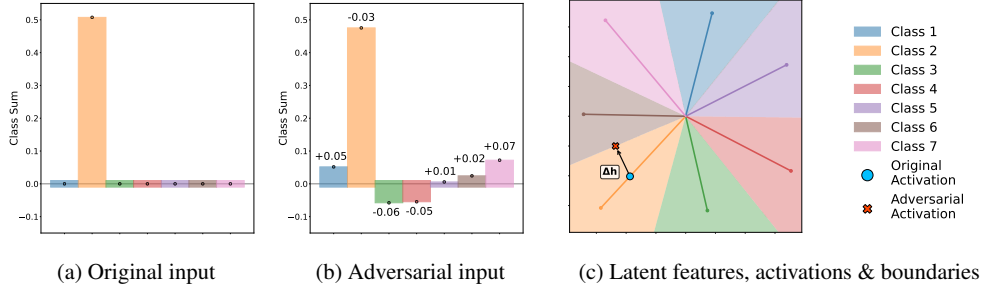


Figure 1: **An adversarial attack exploiting superposition geometry** ($k = 7, m = 2$). (a) Original sample. (b) Adversarially perturbed sample; the ground-truth class is unchanged, but the model prediction is altered. The sign and magnitude of each input perturbation component are determined by the configuration of latent representations. (c) Original and adversarial samples in activation space. Arrows denote the column vectors of $\mathbf{W}_e$, corresponding to the latent representations of input features.

$d = k \times p$ input dimensions into $m < k$ latent dimensions and retain enough information to recover the class with the largest group sum.

In non-synthetic domains, latent features may correspond to nuanced concepts that must be discovered and interpreted post hoc. In contrast, this construction supplies ground-truth class labels for the latent features. We can therefore recompute the true class after any perturbation by recalculating group sums, and define an AEx as a perturbed input satisfying both of the following conditions:

1. The model’s prediction under perturbation changes (*e.g.* from class A to B)
2. The true class remains unchanged (*e.g.* still class A)

This criterion separates genuine AExs from perturbations that alter the ground-truth label itself (*i.e.* change which group has the highest sum). We generate attacks using ℓ_∞ - and ℓ_2 -norm PGD.

Evaluation metrics. We interpret the columns of the encoder weight matrix $\mathbf{W}_e$ as latent representation vectors $\mathbf{v}_i$ for the corresponding input features x_i . Because each input feature $x_i^{(j)}$ contributes only to class j , the vectors $\mathbf{v}_i$ for $i \in \text{class } j$ are expected to align with the class-specific latent direction. This explicit mapping from input coordinates to class-aligned latent directions permits direct measurement of how perturbations applied to inputs of class j , $\mathbf{x}^{(j)}$, propagate into the bottleneck representation. For successful PGD-generated AExs $\mathbf{x}_{adv} = \mathbf{x}_{orig} + \delta$, we measure:

1. **Input perturbation profile (IPP):** The signed magnitude of each δ_i , specifying the coordinate-wise modification applied by the attack.
2. **Latent attack alignment:** The similarity $\Delta \mathbf{h} \cdot \mathbf{v}_c$ between the *latent attack vector* $\Delta \mathbf{h} = \mathbf{h}_{adv} - \mathbf{h}_{orig}$ and the representation of class c .
3. **Attack transferability:** The success rate of attacks generated on a source model when evaluated on a distinct target model.
4. **Robust accuracy:** The fraction of examples that remain correctly classified after applying perturbations δ satisfying $\|\delta\|_p \leq \epsilon$.

3.1 RESULTS

Do attacks exploit interference between latent features in superposition? An attack changes a model prediction by moving the representation of a sample across a decision boundary. The central question is therefore which input-coordinate perturbations implement this boundary crossing. Two baseline intuitions make different predictions: the *feature intuition* predicts that the attack should increase input features associated with the target class, whereas the *bug intuition* predicts that adversarial perturbations are effectively idiosyncratic and cannot be inferred from feature representations.

Fig. 1 shows a representative AEx: an input from Class 2 (orange) is perturbed according to the displayed IPP and is then misclassified as Class 6 (brown). The perturbation does not simply increase the target-class features, contradicting the feature intuition. However, the perturbation is also not arbitrary. Its coordinate-wise structure exhibits a systematic correspondence between δ and the latent feature configuration. This supports a third account: *attacks are mediated by latent geometry*.

Quantitatively, the IPP is strongly coupled to latent attack alignment. The sign and magnitude of perturbations $\delta^{(j)}$ correlate with the alignment between the corresponding class representation $\mathbf{v}^{(j)}$ and the latent attack vector $\Delta\mathbf{h}$. Representations with positive alignment ($\mathbf{v}^{(j)} \cdot \Delta\mathbf{h} > 0$) receive proportional input-feature amplification, whereas negatively aligned representations are attenuated. Thus, Fig. 1(b) visualises input perturbations whose components are determined by the dot products shown in Fig. 1(c), producing movement across the decision boundary.

To test whether PGD specifically exploits superposition geometry, we compare PGD-generated attacks with theoretically optimal perturbations that explicitly depend on superposition interference (derived in Sec. 3.2). For each configuration (k, m) , we train 5 models with distinct random seeds and generate 1000 PGD attacks per model, then compute the cosine similarity between each successful perturbation and its corresponding optimum. As a baseline, we generate random perturbations with matched ℓ_2 norm and measure their similarity to the same optimum. Tab. 1 reports near-perfect alignment between PGD and optimal attacks across dimensionalities. We exclude samples with ℓ_2 norm below ϵ to ensure that measured directions correspond to substantive perturbations. One-sample t-tests comparing observed similarities with the random baseline yield $p < 10^{-10}$ for all configurations, indicating that PGD perturbations match the theoretical directions rather than arising by chance. Because the optimal directions exploit interference and PGD aligns with those directions, we conclude that PGD exploits superposition interference in this setting.

Finding: Adversarial attacks systematically exploit interference between superposed features. Successful PGD attacks are predictable from the specific superposition geometry, rather than being random perturbation patterns.

Do input correlations determine latent feature geometry? We next evaluate whether input-data correlations determine the geometric arrangement of latent features. By systematically modifying the correlation structure of the training distribution, we measure how these correlations constrain the representational degrees of freedom. We use three correlation regimes with increasing constraint strength. In the *uncorrelated* regime, features are sampled independently as above. In the *paired* regime, each feature in class i is coupled to a feature in class j , so that activation of one implies activation of its pair. In the *global* regime, each sample is assigned a random phase that determines a cyclic activation pattern: classes adjacent in index have similar activation probabilities and therefore co-activate frequently, whereas classes separated on the cycle rarely co-activate.

Fig. 2 shows that these correlation regimes determine the arrangement of $\mathbf{v}^{(j)}$. We quantify geometric similarity by computing the pairwise cosine-similarity matrix over all feature pairs in each model and then measuring the correlation between these matrices across random seeds. Larger matrix correlation indicates more similar latent geometry. For each correlation condition, we evaluate 25 seed pairs. In the $m = 2, k = 7$ setup, geometric similarity is 0.23 ± 0.14 (uncorrelated), 0.47 ± 0.32 (paired), and 0.88 ± 0.06 (global) (mean $\pm$ std). These values exhibit a monotonic relation between correlation strength and geometric constraint: uncorrelated data permits highly variable geometries across seeds, paired correlations partially constrain the arrangements, and global correlations produce near-identical geometries. We assess statistical significance using pairwise two-sample t-tests between all three regimes with Bonferroni

Table 1: Alignment between PGD-discovered attacks and theoretically optimal perturbations across configurations. Entries report mean $\pm$ std over 1000 attacks per condition. All p -values are below 10^{-10} .

k	m	Cosine Sim. (PGD vs Theory)	Cosine Sim. (Random Baseline)
6	2	0.97 ± 0.02	0.00 ± 0.02
30	10	0.96 ± 0.00	0.00 ± 0.01
90	30	0.92 ± 0.00	0.00 ± 0.00

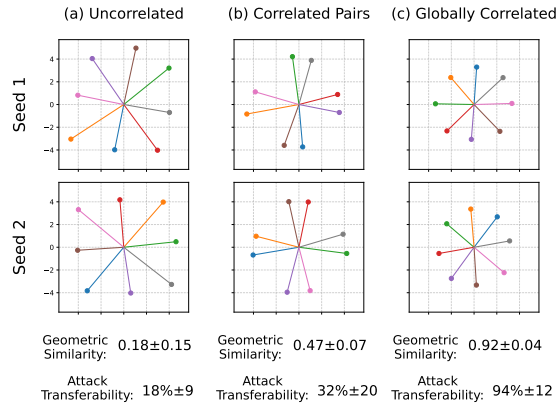


Figure 2: Stronger input correlations induce more consistent feature geometries across initialisations, increasing attack transferability from 18% (uncorrelated) to 94% (global). Error bars denote standard deviations.

correction ($p < 10^{-3}$ for all comparisons, $\alpha = 0.05$ corrected for 3 comparisons). Results for additional (k, m) configurations are reported in Sec. C.3.

The operative mechanism is *interference avoidance*: features that co-activate frequently are arranged to minimise mutual interference. Stronger correlations impose stronger constraints on the feasible geometry. Uncorrelated data permits any arrangement that separates classes; paired correlations constrain the relative positions of coupled features; and global correlations determine a geometry that is effectively unique up to rotation.

Finding: Input correlations constrain feature geometry. Stronger correlations reduce geometric degrees of freedom, forcing different initialisations to converge to similar arrangements.

Does shared geometry explain attack transferability? Having established that input correlations induce consistent latent geometries across model initialisations, we test whether the resulting shared interference patterns account for attack transferability. If adversarial perturbations are specialised to particular interference coefficients, then they should transfer primarily between models whose latent geometries are similar.

Using the same seed pairs as above, we generate adversarial attacks on source models and evaluate their success on target models within each correlation condition, yielding 25 transfer measurements per condition. All models obtain $> 95\%$ clean accuracy. Transfer rates are strongly associated with geometric similarity. For the $k = 7, m = 2$ configuration (Fig. 2), globally correlated data yields $94\% \pm 12\%$ attack transfer (mean $\pm$ std), uncorrelated data yields only $18\% \pm 9\%$ transfer, and paired correlations yield the intermediate value $32\% \pm 20\%$.

Each perturbation component amplifies or suppresses represented features, thereby producing constructive interference that moves the latent representation across a decision boundary. When the same perturbation is applied to a model with a different feature arrangement, components that produced constructive interference in the source model may cancel or interfere destructively in the target model, causing transfer failure.

Finding: Attack transferability is governed by shared interference patterns.

Does reducing superposition suppress these attacks? If superposition induces vulnerability through interference, then reducing or removing superposition should reduce adversarial vulnerability. We test this prediction using three interventions that limit interference. First, we set $m = k$, under which the network learns orthogonal class directions $\mathbf{v}^{(j)}$. In this configuration, any perturbation that changes the prediction also changes the ground-truth class: moving a sample from class A to class B requires the sum of class B features to exceed the sum of class A features, thereby converting the example into a genuine class B sample. Across 1000 attempts at every tested ϵ , we find zero successful adversarial examples. Second, we isolate the effect of superposition from total capacity by fixing m and varying k . Robust accuracy decreases monotonically with superposition pressure k/m (App. Tab. 5), showing that vulnerability scales with the degree of interference. Third, we test whether attacks allocate perturbation budget to superposed features by constraining one class’s latent vector $\mathbf{v}^{(\perp)}$ to remain orthogonal to all other class vectors during training. When attacks are generated between classes that remain in superposition (e.g. from class a to b where $\mathbf{v}_a \not\perp \mathbf{v}_b$), the input features corresponding to the orthogonal class remain unperturbed (Fig. 3).

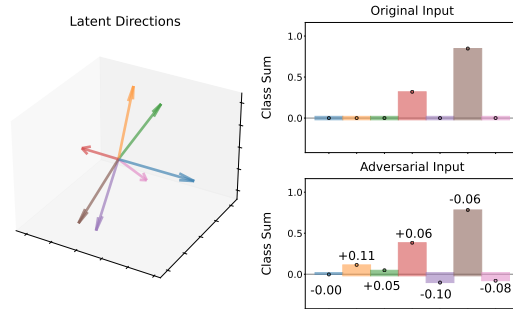


Figure 3: An adversarial attack from class 5 to class 3 does not perturb input features belonging to an orthogonally represented class (class 1).

Finding: Adversarial attacks allocate their perturbation budget to features participating in superposition.

3.2 FORMAL ANALYSIS

We now formalise the mechanism underlying the empirical observations. In the linearised setting, the relation between superposition interference and adversarial vulnerability can be derived exactly, moving the analysis beyond correlational evidence. We use the notation from the experimental setup: $\mathbf{W}_e \in \mathbb{R}^{m \times d}$ induces superposition through columns $\mathbf{v}_i$, and the decoder is taken to correspond to the transposed encoder, as observed empirically. The logit for class j is $z_j = \mathbf{v}_j^\top \mathbf{h}$.

Proposition 1. *The optimal input perturbations δ that maximise movement from class j to class k under constraint $\|\delta\|_2 = \epsilon$ satisfy $\delta \propto \mathbf{W}_e^\top \mathbf{n}$, where $\mathbf{n} = (\mathbf{v}_k - \mathbf{v}_j)$ is the normal to the decision boundary between classes.*

Proof Sketch. To move from class j to k , we maximise the logit margin $z_k - z_j = \mathbf{n}^\top (\mathbf{h} + \Delta \mathbf{h})$, where $\mathbf{n} = (\mathbf{v}_k - \mathbf{v}_j)$ and $\Delta \mathbf{h} = \mathbf{W}_e \delta$. This becomes $\max_{\|\delta\|_2 = \epsilon} \delta^\top \mathbf{W}_e^\top \mathbf{n}$. By Cauchy-Schwarz, this is maximised when $\delta \propto \mathbf{W}_e^\top \mathbf{n}$. Full proof in Sec. B. $\square$

Corollary 1 (Interference Drives Vulnerability) *The adversarial perturbation magnitude for feature i is $|\delta_i| \propto |\mathbf{v}_i^\top (\mathbf{v}_k - \mathbf{v}_j)|$, directly proportional to the differential interference between feature i and the class representations.*

This result specifies how superposition induces adversarial vulnerability. Each input feature i receives perturbation magnitude proportional to its differential interference with the relevant class representations. Under superposition, non-orthogonal feature directions imply that even semantically unrelated features can contribute to the class decision; adversarial perturbations exploit these cross-feature dependencies to control the output.

Proposition 2. *Models with feature representations related by orthogonal transformation $\mathbf{Q}$ (where $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$) share identical optimal attack directions in input space.*

Proof Sketch. Under transformation $\mathbf{v}'_i = \mathbf{Q} \mathbf{v}_i$, optimal perturbations remain invariant: $\delta'_i \propto (\mathbf{v}'_i)^\top (\mathbf{v}'_k - \mathbf{v}'_j) = \mathbf{v}_i^\top \mathbf{Q}^\top \mathbf{Q} (\mathbf{v}_k - \mathbf{v}_j) = \mathbf{v}_i^\top (\mathbf{v}_k - \mathbf{v}_j) = \delta_i$. Full proof in Sec. B. $\square$

Together with the empirical results, these propositions define a **mechanistic pathway**. Specifically: (i) input correlations constrain feature arrangements in superposition, (ii) those geometric arrangements determine interference patterns, (iii) the interference patterns specify optimal perturbations via $\delta \propto \mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j)$ (Proposition 1), and (iv) shared geometric constraints produce similar interference patterns across models, enabling transferability (Proposition 2):

Correlations $\xrightarrow{\text{constrain}}$ **Feature Geometry** $\xrightarrow{\text{determines}}$ **Interference Patterns** $\xrightarrow{\text{enable}}$ **Transferability**

4 ATTACKS IN REALISTIC MODELS

We next evaluate whether the mechanisms identified in the synthetic setting are observable in more realistic vision models, focusing on a ViT (Dosovitskiy et al., 2020) trained on CIFAR-10 (Krizhevsky, 2009) and, in Sec. D.2.1, ResNet-18 (He et al., 2016). We insert a bottleneck layer immediately before the final classification output, thereby forcing latent class representations into an m -dimensional superposition regime. This experiment tests: (i) how the degree of superposition affects attack transferability and robust accuracy, (ii) whether superposed class representations exhibit consistent geometry across initialisations, and (iii) whether the resulting attacks exploit feature interference.

Setup. We train a ViT architecture with 6 layers, patch size 4, and residual-stream dimension $d = 512$ on CIFAR-10 using five random seeds. After training, we replace the original classification head with a bottleneck consisting of a linear encoder from the pre-classification activations to an m -dimensional space, followed by a linear decoder back to the $k = 10$ CIFAR-10 logits. This bottleneck, trained with CE loss and no ReLU to match Sec. 3, constrains the k class representations to occupy an m -dimensional superposed subspace. During bottleneck training, the parameters of the base ViT remain frozen. We vary the bottleneck dimension $m \in \{2, 3, 5, 10\}$ to control the amount of superpositional compression. For each bottlenecked model, we generate AExs using both ℓ_∞ - and ℓ_2 -norm PGD. The attack procedure uses a small step size and many iterations, and terminates once the perturbation changes the model prediction; full hyperparameter details are given in Sec. D.1. We

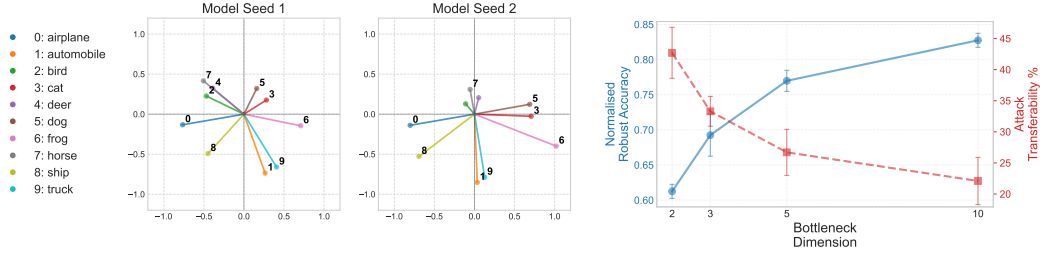


Figure 4: Left: CIFAR-10 class representation geometry remains similar across models trained with different seeds. Right: Attack transferability and robust accuracy as the bottleneck dimension is increased.

normalise robust accuracy by clean accuracy to account for clean-performance differences induced by bottleneck capacity. Attack transferability is computed using the definition in Sec. 3.

Results. The base models achieve mean clean accuracy of 81%. As expected, clean accuracy decreases as bottleneck compression increases: 54%, 67%, 76%, and 81% for $m = \{2, 3, 5, 10\}$.

We observe three patterns that reproduce the synthetic results in this more realistic setting. First, Fig. 4 (right) shows that decreasing the bottleneck dimension m (*i.e.* increasing superposition pressure) simultaneously decreases normalised robust accuracy and increases attack transferability across model initialisations. The robust-accuracy reduction is consistent with denser packing of class representations, which allows small input perturbations to induce broader interference among compressed features. The transferability increase follows the mechanism identified in Sec. 3: stronger superposition reduces the number of feasible latent geometries, so independently initialised models converge to more similar class arrangements and therefore share more interference patterns. Results across perturbation magnitudes are provided in Sec. D.1.1.

Second, Fig. 4 (left) shows that the m -dimensional class representations in the bottleneck exhibit consistent ordering and clustering across models. As in Sec. 3, this consistency suggests that correlations in the data constrain the formation of superposed geometry. These clusters often align with semantic class similarity (*e.g.* ‘cat’ and ‘dog’ representations appearing near one another). Third, when AExs generated for a bottlenecked model are evaluated through the corresponding base ViT, they induce similar relative logit changes (*i.e.* $\text{logit}_{\text{base}}(\mathbf{x}_{\text{adv}}) - \text{logit}_{\text{base}}(\mathbf{x}_{\text{orig}})$). Thus, the base and bottlenecked models respond coherently to the same perturbation directions. The perturbations are therefore not solely artefacts of the inserted bottleneck, although the uncompressed base model does not misclassify under those perturbations.

5 ADVERSARIAL VULNERABILITY VIA ALGORITHMIC BRITTLINESS

The preceding sections show that feature superposition can *suffice* for adversarial vulnerability. We now test whether superposition is *necessary* by analysing a setting in which the relevant features are represented orthogonally rather than in superposition, yet the model remains adversarially vulnerable. This section supports the broader claim that feature-representation analysis is useful for robustness analysis beyond superposition itself. We address three questions: *what is the mechanism of these attacks, can mechanistic knowledge be used to construct attacks directly without gradients, and does naive robustness certification change the failure mode or only increase the required attack budget?*

Setup. We train a linear encoder followed by an MLP on one-hot encoded integer pairs (a, b) to predict $(a + b) \bmod P$. As described in Sec. E and Nanda et al. (2023), the model learns a trigonometric algorithm for this task. Within this learned algorithm¹, the encoder maps inputs to non-basis-aligned linear representations encoding $\sin(\omega_k a)$ and $\cos(\omega_k a)$ at key frequencies $\omega_k = 2\pi k/P$. The MLP then composes $\cos((a + b)\omega_k)$ using sum-angle identities and reaches 100% accuracy. Crucially, these frequency features are represented *orthogonally* (*i.e.* not in superposition). We generate AExs by relaxing the one-hot input constraint and applying PGD only to input a , thereby moving examples off the training manifold.

¹We refer to the specific computation executed by the network to solve the task as the *model’s learned algorithm*. As this algorithm depends on extracting precise values from learned feature representations (frequencies), it remains *brittle* in the face of small perturbations to the bases. We term this the *algorithmic brittleness*.

Attacks exploit the learned key frequencies. Fourier analysis of successful perturbations δ (App. Fig. 6) shows that their dominant frequencies match the learned algorithm’s key frequencies, ω_k . Attacks exhibit 0% transfer to models with different ω_k , *e.g.* models trained from different seeds. These frequency-aligned perturbations induce misclassification by changing the sinusoidal values read from the encoder features. The perturbation δ maps the one-hot vector $\mathbf{x}_a$ to a dense vector $\mathbf{x}'_a$. The newly non-zero coordinates contribute additional sinusoidal components, corrupting the values used by the MLP and thereby disrupting the downstream computation. Because the model’s algorithm aggregates the constructive contribution of each ω_k at the logits, perturbing a single ω_k is insufficient. Successful attacks must therefore find δ that modifies multiple frequency components simultaneously. We provide a more detailed mechanistic intuition in Sec. E.1.

Analytic construction of attacks. This mechanism enables gradient-free, *informed* perturbations constructed as composite sinusoidal waves with matching ω_k . On $0 + 0 \bmod P$, these attacks succeed with $\|\delta\|_2=0.22$, close to PGD (0.17, $\approx 1.3\times$ the PGD budget) and substantially below naive alternatives (single-index 0.97, ℓ_∞ noise 2.04, uniform 6.20). This result provides empirical evidence that the attack targets the identified mechanism and shows that representation-aware vulnerability analysis can produce concrete attack constructions, rather than only post-hoc explanations.

Do defences mitigate this algorithmic vulnerability? We apply certified training against ℓ_∞ perturbations (Müller et al., 2023). For fixed attack budgets, *robust* accuracy improves from the pre-robustified model to the post-robustified model as follows: 0% $\rightarrow$ 85% at $\epsilon_\infty = 5e-3$, 0% $\rightarrow$ 96% at $2e-3$, and 76% $\rightarrow$ 100% at $1e-3$. However, under PGD, the robust model still fails through the same frequency-based brittleness, although larger perturbation norms are required. Certification therefore increases the attack budget required for failure without eliminating the specific algorithmic fragility, motivating defences that target the identified semantics of the computation (here, frequency manipulation) rather than relying only on broad norm constraints.

Relation to the paper’s thesis. This section establishes two claims that integrate with the paper’s broader argument. First, superposition is *sufficient but not necessary*: orthogonal representations can also be vulnerable to attack. Second, analysing AExs as an *architecture–semantics interaction* is operationally useful: it can guide both attack design through representation-informed perturbations and defence design through mechanism-specific interventions.

Although not central to this section, the mechanism also depends on sparsity assumptions: upstream sparsity violations (*e.g.* noise or superposed features) can provide effectively ‘pre-perturbed’ inputs to downstream orthogonal circuits, thereby activating the same frequency-based failure mode.

6 RELATED WORK

Superposition & latent representations. Prior work on superposition has identified several mechanisms by which feature directions interfere. Nanda (2023) distinguish *representational* superposition from *computational* superposition, while Gurnee et al. (2023) distinguish *alternating* from *simultaneous* interference. Data correlations are also known to constrain representational geometry: correlated feature pairs can become orthogonal (Elhage et al., 2022), correlation structure can drive the formation of superposition (Chan, 2024), and semantic correlations can induce clustering (Prieto et al., 2025a). More broadly, representation geometry is affected by spectral bias (Rahaman et al., 2019), neural collapse (Kothapalli, 2023), and optimisation objectives (Casper, 2023). Sparse autoencoders (SAEs) have been applied to vision models as tools for separating features represented in superposition (Lim et al., 2024; Joseph et al., 2025).

Adversarial vulnerability. The adversarial-robustness literature is commonly organised around attacks and defences. Attacks (Szegedy et al., 2014) construct small perturbations that maximise model error, with standard procedures including FGSM (Goodfellow et al., 2014) and PGD (Madry et al., 2018). Adversarial robustness denotes the preservation of predictive performance under such perturbations. Defence methods include adversarial training (Madry et al., 2018) and certified training (Wong & Kolter, 2018; Palma et al., 2024), the latter providing formal guarantees within specified perturbation sets.

The role of input & latent features on adversarial robustness. Several accounts attribute vulnerability to predictive but brittle ‘non-robust’ input features (Ilyas et al., 2019; Goh, 2019) or to learned shortcuts (Li et al., 2023). Perturbation structure can also vary substantially across datasets (Maiya et al., 2021). Elhage et al. (2022) propose that superposition may be connected to AExs,

whereas Casper (2023) discuss limitations of that connection. Recent work also observes a tension between representational efficiency and adversarial robustness (Barsbey et al., 2025). Zhang et al. (2021) show that adversarial perturbations move latent representations across decision boundaries, and Ganeshan et al. (2019) find that PGD targets final layers and motivate feature-based attacks. Attack transferability is limited by representational discrepancies between models (Li et al., 2023; Wang et al., 2024); conversely, decorrelating features reduces transfer (Wiedeman & Wang, 2022), consistent with our geometric account.

7 DISCUSSION & CONCLUDING REMARKS

We have shown that adversarial attacks can exploit interference patterns determined by the geometry of superposed features in NNs. Across synthetic models and ViTs, the experiments indicate that data properties—specifically correlations and sparsity—constrain superposition geometry, and that this geometry induces predictable adversarial vulnerabilities. The resulting interference structure explains phenomena including attack transferability and class-specific susceptibility. This perspective frames adversarial vulnerability as a possible consequence of efficient representational compression via superposition, rather than solely as an anomaly of optimisation or data.

Limitations & future work. The analysis is primarily derived from simplified and partially engineered settings. Although these experiments provide an initial mechanistic lens for attacks in larger models, the broader adversarial-robustness literature suggests that the relevant vulnerability mechanism may depend on the attack class, architecture, and data modality. For example, constrained attack types (*e.g.* universal perturbations, jailbreaks, patch attacks) may not be able to activate the specific feature combinations required to exploit superposition interference.

Future work should quantify the ‘adversarial cost’ of superposition, characterise its relation to accuracy-robustness trade-offs, and determine how robust training changes superposition geometry. The present results rely on settings in which class features are placed in engineered superposition, whereas real networks contain unknown and unlabelled features that may interfere across multiple layers. We plan to use SAEs (Bricken et al., 2023) to address these challenges in large-scale networks, with preliminary results in Sec. F suggesting that this is a promising approach.

ETHICS STATEMENT

Because the work is primarily mechanistic and explanatory, we do not anticipate specific negative impacts or immediate societal consequences from the proposed framework. The experiments use only publicly available data (CIFAR-10) and synthetic toy models, and they do not involve human subjects or personally identifiable information. The gradient-based attack procedures used in the paper are standard methods from the existing literature. The work does not introduce a qualitatively new attack capability; instead, it provides a technical account of previously observed adversarial phenomena. We have no conflicts of interest to declare. By clarifying the relationship between superposition and adversarial vulnerability, we aim to inform future research on balancing model capability with robustness and on developing trustworthy AI systems for safety-critical deployment contexts.

REPRODUCIBILITY STATEMENT

We provide source code for all experiments in the supplementary materials. The toy-model experiments (Sec. 3) specify the architectures, training procedures, evaluation metrics, and data-generation processes. The ViT experiments (Sec. 4) report the architecture and training procedure, with additional implementation details in Sec. D.1. The modular-arithmetic experiments (Sec. 5) specify the network architecture, data encoding, and learned trigonometric algorithm, with further details in Sec. E.2. Experiments were conducted on an AMD Ryzen Threadripper 3970X 32-Core with 256GB RAM and RTX 3090. Additional ResNet-18 and SAE experiments are documented in Appendices Sec. D.2.1 and Sec. F, respectively.

LLM USAGE STATEMENT

This work used LLMs to assist with prior literature search, automate LaTeX table generation from CSV files (with all numerical values manually verified), and suggest alternative phrasings for manuscript clarity. All substantive research contributions, analyses, and conclusions remain the original work of the authors.

REFERENCES

- Melih Barsbey, Antônio H Ribeiro, Umut Şimşekli, and Tolga Birdal. On the interaction of compressibility and adversarial robustness. *arXiv preprint arXiv:2507.17725*, 2025.
- Leonard Bereska and Stratis Gavves. Mechanistic interpretability for AI safety - a review. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856.
- Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.
- Sébastien Bubeck and Mark Sellke. A universal law of robustness via isoperimetry. In Marc’Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan (eds.), *Advances in Neural Information Processing Systems*, pp. 28811–28822, 2021.
- Stephen Casper. EIS IX: Interpretability and adversaries. *AI Alignment Forum*, February 2023.
- Lawrence Chan. Superposition is not “just” neuron polysemanticity. *AI Alignment Forum*, April 2024.
- Joana C Costa, Tiago Roxo, Hugo Proença, and Pedro Ricardo Morais Inacio. How deep learning sees the world: A survey on adversarial attacks & defenses. *IEEE Access*, 12:61113–61136, 2024.
- Valérie Costa, Thomas Fel, Ekdeep Singh Lubana, Bahareh Tolooshams, and Demba Ba. From flat to hierarchical: Extracting sparse representations with matching pursuit. *arXiv preprint arXiv:2506.03093*, 2025.

594 Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hier-
595 archical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*,
596 pp. 248–255, 2009.

597 Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas
598 Unterthiner, Mostafa Dehghani, Matthias Minderer, G Heigold, S Gelly, et al. An image is
599 worth 16x16 words: Transformers for image recognition at scale. In *International Conference on*
600 *Learning Representations*, 2020.

601 Rüdiger Ehlers. Formal verification of piece-wise linear feed-forward neural networks. In Deepak
602 D’Souza and K. Narayan Kumar (eds.), *Automated Technology for Verification and Analysis -*
603 *15th International Symposium, ATVA 2017, Pune, India, October 3-6, 2017, Proceedings*, volume
604 10482 of *Lecture Notes in Computer Science*, pp. 269–286. Springer, 2017.

605 Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec,
606 Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition.
607 *arXiv preprint arXiv:2209.10652*, 2022.

608 Alhussein Fawzi, Seyed-Mohsen Moosavi-Dezfooli, and Pascal Frossard. Robustness of classifiers:
609 from adversarial to random noise. *Advances in neural information processing systems*, 29, 2016.

610 Aditya Ganeshan, Vivek BS, and R Venkatesh Babu. FDA: Feature disruptive attack. In *Proceedings*
611 *of the IEEE/CVF International Conference on Computer Vision*, pp. 8069–8079, 2019.

612 Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever,
613 Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. In *The Thirteenth*
614 *International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*,
615 2025.

616 Gabriel Goh. A discussion of ‘adversarial examples are not bugs, they are features’: Two examples
617 of useful, non-robust features. *Distill*, 2019.

618 Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial
619 examples. *arXiv preprint arXiv:1412.6572*, 2014.

620 Clément Guerner, Anej Svete, Tianyu Liu, Alexander Warstadt, and Ryan Cotterell. A geometric
621 notion of causal probing. *arXiv preprint arXiv:2307.15054*, 2023.

622 Wes Gurnee, Neel Nanda, Matthew Pauly, Katherine Harvey, Dmitrii Troitskii, and Dimitris Bertsimas.
623 Finding neurons in a haystack: Case studies with sparse probing. *Trans. Mach. Learn. Res.*, 2023.

624 Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image
625 recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*,
626 pp. 770–778, 2016.

627 Matthias Hein and Maksym Andriushchenko. Formal guarantees on the robustness of a classi-
628 fier against adversarial manipulation. In *Advances in Neural Information Processing Systems*,
629 volume 30, 2017.

630 Patrick Henriksen and Alessio Lomuscio. Robust training of neural networks against bias field
631 perturbations. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(12):14865–14873,
632 Jun. 2023.

633 Nikolaus Howe, Ian McKenzie, Oskar Hollinsworth, Michał Zajac, Tom Tseng, Aaron Tucker,
634 Pierre-Luc Bacon, and Adam Gleave. Scaling trends in language model robustness, 2025.

635 A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, and A. Madry. Adversarial examples are not
636 bugs, they are features. In *Advances in Neural Information Processing Systems 32*, pp. 125–136,
637 2019.

638 Sonia Joseph, Praneet Suresh, Lorenz Hufe, Edward Stevinson, Robert Graham, Yash Vadi, Danilo
639 Bzdok, Sebastian Lapuschkin, Lee Sharkey, and Blake Aaron Richards. Prisma: An open source
640 toolkit for mechanistic interpretability in vision and video, 2025.

-
- Guy Katz, Clark W. Barrett, David L. Dill, Kyle Julian, and Mykel J. Kochenderfer. Reluplex: An efficient SMT solver for verifying deep neural networks. In Rupak Majumdar and Viktor Kuncak (eds.), *Computer Aided Verification - 29th International Conference*, volume 10426, pp. 97–117. Springer, 2017.
- Vignesh Kothapalli. Neural collapse: A review on modelling principles and generalization. *Trans. Mach. Learn. Res.*, 2023.
- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. CIFAR-10 dataset.
- Victor Lecomte, Kushal Thaman, Rylan Schaeffer, Naomi Bashkansky, Trevor Chow, and Sanmi Koyejo. What causes polysemanticity? an alternative origin story of mixed selectivity from incidental causes. *arXiv preprint arXiv:2312.03096*, 2023.
- Ang Li, Yifei Wang, Yiwen Guo, and Yisen Wang. Adversarial examples are not real features. *Advances in Neural Information Processing Systems*, 2023.
- Hyesu Lim, Jinho Choi, Jaegul Choo, and Steffen Schneider. Sparse autoencoders reveal selective remapping of visual concepts during adaptation. *arXiv preprint arXiv:2412.05276*, 2024.
- Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. On the biology of a large language model. *Transformer Circuits Thread*, 2025.
- Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards Deep Learning Models Resistant to Adversarial Attacks. In *International Conference on Learning Representations*, February 2018.
- Shishira R. Maiya, Max Ehrlich, Vatsal Agarwal, Ser-Nam Lim, Tom Goldstein, and Abhinav Shrivastava. A Frequency Perspective of Adversarial Robustness. *arXiv*, 2021.
- Mark Niklas Müller, Franziska Eckert, Marc Fischer, and Martin T. Vechev. Certified training: Small boxes are all you need. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- Preetum Nakkiran. A discussion of ‘adversarial examples are not bugs, they are features’: Adversarial examples are just bugs, too. *Distill*, 2019.
- Neel Nanda. A comprehensive mechanistic interpretability explainer and glossary, 2023.
- Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- Alessandro De Palma, Rudy Bunel, Krishnamurthy Dvijotham, M. Pawan Kumar, and Robert Stanforth. IBP regularization for verified adversarial robustness via branch-and-bound. *CoRR*, abs/2206.14772, 2022.
- Alessandro De Palma, Rudy Bunel, Krishnamurthy (Dj) Dvijotham, M. Pawan Kumar, Robert Stanforth, and Alessio Lomuscio. Expressive losses for verified robustness via convex combinations. In *The Twelfth International Conference on Learning Representations, ICLR, 2024*.
- Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024.
- L Prieto, E Stevinson, M Barsbey, T Birdal, and P Mediano. Correlations in the data lead to semantically rich feature geometry under superposition. *Mechanistic Interpretability Workshop at NeurIPS 2025*, 2025a.

-
- Lucas Prieto, Melih Barsbey, Pedro A. M. Mediano, and Tolga Birdal. Grokking at the edge of numerical stability. In *The Thirteenth International Conference on Learning Representations, ICLR, 2025b*.
- Aditi Raghunathan, Jacob Steinhardt, and Percy Liang. Certified defenses against adversarial examples. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018.
- Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron C. Courville. On the spectral bias of neural networks. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, volume 97 of *Proceedings of Machine Learning Research*, pp. 5301–5310. PMLR, 2019.
- Shashata Sawmya, Linghao Kong, Ilia Markov, Dan Alistarh, and Nir N Shavit. Wasserstein distances, neuronal entanglement, and sparsity. In *The Thirteenth International Conference on Learning Representations, 2025*.
- Adam Scherlis, Kshitij Sachan, Adam S Jermyn, Joe Benton, and Buck Shlegeris. Polysemanticity and capacity in neural networks. *arXiv preprint arXiv:2210.01892*, 2022.
- Ludwig Schmidt, Shibani Santurkar, Dimitris Tsipras, Kunal Talwar, and Aleksander Madry. Adversarially robust generalization requires more data. In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pp. 5019–5031, 2018.
- C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. Intriguing properties of neural networks. In *2nd International Conference on Learning Representations, ICLR 2014, 2014*.
- Tomasz Tkocz. An upper bound for spherical caps. *The American Mathematical Monthly*, 119(7): 606–607, 2012.
- Donghua Wang, Wen Yao, Tingsong Jiang, Xiaohu Zheng, Junqi Wu, and Xiaoqian Chen. Improving the Transferability of Adversarial Examples by Feature Augmentation. *arXiv*, 2024.
- Christopher Wiedeman and Ge Wang. Disrupting adversarial transferability in deep neural networks. *Patterns*, 3(5), 2022.
- Eric Wong and Zico Kolter. Provable defenses against adversarial examples via the convex outer adversarial polytope. In *International conference on machine learning*. PMLR, 2018.
- Huan Zhang, Hongge Chen, Chaowei Xiao, Sven Gowal, Robert Stanforth, Bo Li, Duane S. Boning, and Cho-Jui Hsieh. Towards stable and efficient training of verifiably robust neural networks. In *8th International Conference on Learning Representations, ICLR, 2020*.
- Shufei Zhang, Zhuang Qian, Kaizhu Huang, Qiufeng Wang, Rui Zhang, and Xinping Yi. Towards better robust generalization with shift consistency regularization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pp. 12524–12534. PMLR, 2021.

APPENDIX

A GENERATION OF ADVERSARIAL EXAMPLES

We generate AExs using PGD, an iterative optimisation procedure for constructing adversarial perturbations (Madry et al., 2018). For untargeted attacks, the procedure maximises the classifier loss $\mathcal{L}(f(\mathbf{x}'), y_{\text{true}})$; for targeted attacks, it minimises $\mathcal{L}(f(\mathbf{x}'), y_{\text{target}})$ according to the update rule:

$$\mathbf{x}'^{(k+1)} = \Pi_{\mathcal{S}} \left(\mathbf{x}'^{(k)} \pm \alpha \mathbf{g}_k \right) \quad (1)$$

Here $+$ denotes maximisation and $-$ denotes minimisation. $\mathbf{x}'^{(0)}$ is the initial perturbed input, α is the step size, $\mathbf{g}_k$ is the normalised gradient $\nabla_{\mathbf{x}} \mathcal{L}(f(\mathbf{x}'^{(k)}), y_{\text{class}})$ with y_{class} equal to either y_{true} or y_{target} , and $\Pi_{\mathcal{S}}$ projects onto the ϵ -ball $\mathcal{S} = \{\mathbf{x}' \mid \|\mathbf{x}' - \mathbf{x}\|_p \leq \epsilon\}$ with input-domain clipping. We use the following ℓ_p constraints:

1. ℓ_{∞} : The constraint is $\|\delta\|_{\ell_{\infty}} = \max_i |\delta_i| \leq \epsilon$. The update direction $\mathbf{g}_k$ is $\text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\cdot))$, and $\Pi_{\mathcal{S}}$ clips each coordinate $\mathbf{x}'_i$ to $[\mathbf{x}_i - \epsilon, \mathbf{x}_i + \epsilon]$.
2. ℓ_2 : The constraint is $\|\delta\|_{\ell_2} = \sqrt{\sum_i \delta_i^2} \leq \epsilon$. The update direction $\mathbf{g}_k$ is $\nabla_{\mathbf{x}} \mathcal{L}(\cdot) / \|\nabla_{\mathbf{x}} \mathcal{L}(\cdot)\|_{\ell_2}$. The projection $\Pi_{\mathcal{S}}$ rescales $\delta = \mathbf{x}' - \mathbf{x}$ whenever $\|\delta\|_{\ell_2} > \epsilon$ according to $\delta \leftarrow \epsilon \cdot \delta / \|\delta\|_{\ell_2}$.

B OMITTED PROOFS & THEORETICAL RESULTS

To characterise how adversarial attacks exploit feature representations, we prove that optimal perturbations weight each input coordinate according to the alignment between its associated feature direction and the normal direction to the relevant decision boundary. The analysis is carried out for the linear model without ReLU activation functions.

Consider an input $\mathbf{x} \in \mathbb{R}^d$ encoded as $\phi(\mathbf{x}) = \mathbf{W}_e \mathbf{x}$, producing latent representation $\mathbf{h} \in \mathbb{R}^m$. The columns of $\mathbf{W}_e \in \mathbb{R}^{m \times d}$ form an overcomplete dictionary $\{\mathbf{v}_i\}_{i=1}^d$ with $m < d$. For the argmax task, we empirically observe that the trained encoder-decoder pair satisfies $\mathbf{W}_d = \mathbf{W}_e^{\top}$. Thus, the logit for class j is $z_j = \mathbf{v}_j^{\top} \mathbf{h}$, where $\mathbf{v}_j$ is the j -th column of $\mathbf{W}_e$, and the predicted class is $\hat{y} = \arg \max_j z_j$. Input perturbations induce latent perturbations by $\Delta \mathbf{h} = \mathbf{W}_e \delta$, where $\delta = (\delta_1, \dots, \delta_d)^{\top}$ denotes the vector of perturbation coefficients.

The pairwise decision boundary between classes j and k is the set of latent points at which the two logits are equal: $\mathcal{B}_{jk} = \{\mathbf{h} : z_j = z_k\}$, equivalently $\{\mathbf{h} : (\mathbf{v}_k - \mathbf{v}_j)^{\top} \mathbf{h} = 0\}$. The vector $\mathbf{n} = (\mathbf{v}_k - \mathbf{v}_j)$ is the normal to this boundary and points in the direction that maximally increases the margin $z_k - z_j$. We restate the propositions before giving their proofs.

Proposition 1 (Optimal targeted attack). The optimal input perturbations δ that maximise movement from class j toward class k under constraint $\|\delta\|_2 = \epsilon$ satisfy:

$$\delta \propto \mathbf{W}_e^{\top} \mathbf{n}$$

Proof. To move a sample from class j toward class k , the perturbation must maximise the logit margin $z_k - z_j$. Under perturbation δ , the updated margin is:

$$z'_k - z'_j = \mathbf{v}_k^{\top} (\mathbf{h} + \Delta \mathbf{h}) - \mathbf{v}_j^{\top} (\mathbf{h} + \Delta \mathbf{h}) \quad (2)$$

$$= (\mathbf{v}_k - \mathbf{v}_j)^{\top} (\mathbf{h} + \mathbf{W}_e \delta) \quad (3)$$

$$= (\mathbf{v}_k - \mathbf{v}_j)^{\top} \mathbf{h} + (\mathbf{v}_k - \mathbf{v}_j)^{\top} \mathbf{W}_e \delta \quad (4)$$

The induced margin change is $(\mathbf{v}_k - \mathbf{v}_j)^{\top} \mathbf{W}_e \delta = \delta^{\top} \mathbf{W}_e^{\top} (\mathbf{v}_k - \mathbf{v}_j)$. The optimisation problem is therefore:

$$\max_{\delta} \delta^{\top} \mathbf{W}_e^{\top} (\mathbf{v}_k - \mathbf{v}_j) \quad \text{subject to} \quad \|\delta\|_2 = \epsilon$$

Let $\mathbf{g} = \mathbf{W}_e^{\top} (\mathbf{v}_k - \mathbf{v}_j)$. By the Cauchy-Schwarz inequality:

$$|\delta^{\top} \mathbf{g}| \leq \|\delta\|_2 \|\mathbf{g}\|_2 = \epsilon \|\mathbf{g}\|_2$$

Equality is achieved when δ and $\mathbf{g}$ are parallel. Given the constraint $\|\delta\|_2 = \epsilon$:

$$\delta = \frac{\epsilon}{\|\mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j)\|_2} \mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j)$$

□

Corollary (Interference drives vulnerability). For a targeted attack from class j to class k , the adversarial perturbation magnitude for input feature i is:

$$|\delta_i| \propto |\mathbf{v}_i^\top (\mathbf{v}_k - \mathbf{v}_j)|$$

where $\mathbf{v}_i$ is the i -th column of $\mathbf{W}_e$. This quantity represents the differential interference between feature i and the class representations.

This reveals the mechanism by which superposition creates adversarial vulnerability. Each input feature i is perturbed proportionally to $\mathbf{v}_i^\top (\mathbf{v}_k - \mathbf{v}_j)$ —the differential interference between feature i and the class representations. Under superposition, the non-orthogonality means that even semantically unrelated features have non-zero inner products with $(\mathbf{v}_k - \mathbf{v}_j)$, creating exploitable interference patterns. The multi-class setting amplifies this vulnerability, as with k classes there are multiple possible pairwise boundaries, each creating a distinct interference pattern. Adversarial perturbations leverage these cross-feature dependencies—they manipulate features that affect it through their interference with the class representations. This explains the vulnerability we observe empirically: attacks succeed not by directly increasing target class features, but by exploiting the web of interference created by superposition.

Prop. 1 characterises the optimal perturbation direction for moving from class j to k , providing the gradient for maximising the margin $z_k - z_j$. When intervening classes exist (where $z_j < z_i < z_k$ for some class i), following this gradient might cause the model to predict i before reaching target k . Iterative methods like PGD handle this by recomputing gradients at each step—the global attack path emerges from repeated local decisions rather than a single optimisation. Furthermore, while our analysis assumes $\mathbf{W}_d = \mathbf{W}_e^\top$ based on our empirical observations, the framework extends to arbitrary decoders. In the general case, with decoder $\mathbf{W}_d \in \mathbb{R}^{k \times m}$ having rows $\mathbf{w}_j^\top$, the optimal perturbation becomes $\delta \propto \mathbf{W}_e^\top (\mathbf{w}_k - \mathbf{w}_j)$, capturing interference between encoder directions and decoder weights.

Proposition 2 (Attack transferability). Consider encoders ϕ and ψ with basis matrices $\mathbf{W}_e \in \mathbb{R}^{m \times d}$ and $\mathbf{W}'_e \in \mathbb{R}^{m \times d}$ whose columns are related by orthogonal transformation $\mathbf{v}'_i = \mathbf{Q}\mathbf{v}_i$ (where $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$). If both models use their encoder transpose as decoder (i.e., $\mathbf{W}_d = \mathbf{W}_e^\top$ and $\mathbf{W}'_d = (\mathbf{W}'_e)^\top$), then both models have identical optimal input perturbation vectors for any targeted attack from class j to class k .

Proof. For model ϕ , the optimal perturbation from Prop. 1 is:

$$\delta^\phi \propto \mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j)$$

For model ψ with transformed columns $\mathbf{v}'_i = \mathbf{Q}\mathbf{v}_i$:

$$\delta^\psi \propto (\mathbf{W}'_e)^\top (\mathbf{v}'_k - \mathbf{v}'_j) = (\mathbf{W}'_e)^\top (\mathbf{Q}\mathbf{v}_k - \mathbf{Q}\mathbf{v}_j)$$

Since $\mathbf{W}'_e = \mathbf{Q}\mathbf{W}_e$ (all columns are transformed), we have:

$$\delta^\psi \propto (\mathbf{Q}\mathbf{W}_e)^\top \mathbf{Q}(\mathbf{v}_k - \mathbf{v}_j) \tag{5}$$

$$= \mathbf{W}_e^\top \mathbf{Q}^\top \mathbf{Q}(\mathbf{v}_k - \mathbf{v}_j) \tag{6}$$

$$= \mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j) \tag{7}$$

where the last equality uses $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$. Thus δ^ϕ and δ^ψ have identical proportionality. Under the same norm constraint $\|\delta\|_2 = \epsilon$, we have $\delta^\phi = \delta^\psi$. □

Prop. 2 is used to explain why attacks transfer between models with similar training regimes. When models learn feature representations that differ only by orthogonal transformation—essentially the

same geometric structure in different orientations—they share identical vulnerability patterns in input space. The orthogonal transformation preserves all inner products between features, maintaining the interference patterns that attacks exploit. This provides an explanation for our empirical observation that models trained on data with the same correlations exhibit high attack transferability: they discover similar feature geometries up to rotation, leading to shared adversarial vulnerabilities.

C TOY MODEL EXPERIMENTS

This section provides supplementary experimental details, extended quantitative results, and additional discussion for the toy-model experiments in the main paper. We report model accuracies across a broader parameter range than in the main text, clarifying how bottleneck capacity and data properties such as sparsity affect learning dynamics and the emergence of feature superposition. We then provide additional visual examples corresponding to Fig. 1, illustrating the attack mechanism under several configurations.

C.1 HYPOTHESES TESTING FRAMEWORK

We formalise the hypotheses associated with the three research questions in Sec. 3.

Research Q1: Do adversarial perturbations exploit superposition geometry?

- H_0 : Adversarial perturbations are random with respect to feature geometry.
- H_1 : Adversarial perturbations systematically exploit geometric relationships between superposed representations.

Research Q2: Do data correlations determine superposition geometry?

- H_0 : Input correlations have no systematic effect on learned geometries.
- H_1 : Input correlations determine geometric arrangements across model initializations.

Research Q3: Does shared geometry explain attack transferability?

- H_0 : Attack transferability is independent of geometric similarity.
- H_1 : Transferability increases with shared latent structure.

We evaluate these hypotheses using the following controlled measurements:

- $H_1(1)$: We measure the input perturbation profile alignment with a class’s latent representation and the latent attack vector.
- $H_1(2)$: We systematically vary input correlations and measure resulting geometries.
- $H_1(3)$: We quantify transferability rates across models with varying geometric similarity.

C.2 ACCURACY OF TOY MODEL FOR A RANGE OF PARAMETERS

The toy-model experiments in the main paper primarily use low-dimensional settings for interpretability. To characterise the model more broadly, this subsection reports classification accuracies for the CE toy model across hidden size (h), number of classes (k), number of features, and sparsity level (S). These results specify when the model can learn useful superposed representations under varying compression and sparsity regimes. The sparsity level is the probability that an input feature $x_i^{(j)}$ is set to zero; larger S therefore corresponds to sparser inputs. We report results as a function of feature density $1 - S$, the probability that a feature is non-zero. Tab. 2 and Tab. 3 contextualise the model’s performance limits and its ability to learn latent representations in superposition.

C.3 CORRELATIONS CONSTRAIN ARRANGEMENTS

Tab. 4 reports the extended results for the question *Do Input Correlations Determine Latent Feature Geometry?* across additional numbers of classes (k) and hidden dimensions (m).

Table 2: Classification accuracy of the CE toy model with a fixed bottleneck dimension ($m = 2$) across various numbers of classes (k), total input features (features = $k \times 3$), and input feature densities ($1 - S$). These results illustrate how input sparsity controls performance degradation as the number of classes to be superposed within a constrained latent space increases.

Classes (k)	Features	Hidden (m)	Accuracy at Input Feature Density ($1 - S$)							
			1.0	0.57	0.33	0.19	0.11	0.06	0.04	0.02
3	9	2	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
4	12	2	0.67	0.71	0.82	0.92	0.98	0.99	1.00	1.00
5	15	2	0.53	0.50	0.65	0.77	0.89	0.95	0.98	0.99
6	18	2	0.40	0.43	0.51	0.66	0.82	0.93	0.97	0.99
7	21	2	0.34	0.34	0.40	0.53	0.73	0.87	0.95	0.98
8	24	2	0.30	0.30	0.33	0.40	0.63	0.82	0.93	0.97
9	27	2	0.24	0.26	0.30	0.35	0.57	0.75	0.89	0.96
10	30	2	0.22	0.24	0.26	0.31	0.50	0.72	0.87	0.95
15	45	2	0.14	0.15	0.16	0.17	0.25	0.40	0.65	0.86
20	60	2	0.10	0.10	0.11	0.13	0.15	0.24	0.44	0.76
25	75	2	0.07	0.08	0.09	0.10	0.12	0.16	0.26	0.62
30	90	2	0.06	0.07	0.07	0.07	0.09	0.11	0.21	0.45

Table 3: Classification accuracy of the CE toy model for varying numbers of classes (k), total input features, bottleneck dimensions (m), and input feature sparsity levels (S). This table reports different numbers of features per class (p).

Classes (k)	Features	Hidden (m)	Accuracy at Input Feature Density ($1 - S$)							
			1.0	0.57	0.33	0.19	0.11	0.06	0.04	0.02
30	30	30	0.23	0.24	0.38	0.62	0.83	0.94	0.99	1.00
30	90	90	0.27	0.27	0.33	0.41	0.51	0.67	0.85	0.94
40	40	30	0.67	0.54	0.73	0.77	0.88	0.96	0.99	0.99
40	120	30	0.71	0.64	0.65	0.72	0.73	0.79	0.89	0.96
60	60	10	0.05	0.07	0.12	0.25	0.47	0.73	0.90	0.97
60	180	10	0.08	0.10	0.13	0.17	0.22	0.32	0.52	0.75
80	80	30	0.15	0.17	0.23	0.41	0.63	0.79	0.91	0.98
80	240	30	0.23	0.22	0.31	0.41	0.48	0.53	0.66	0.80
100	100	10	0.03	0.04	0.05	0.10	0.21	0.43	0.69	0.87
100	300	10	0.04	0.05	0.06	0.09	0.12	0.15	0.25	0.45

Table 4: Geometric similarity across correlation types and superposition configurations.

Correlation Type	k	m	Geometric Similarity
Uncorrelated	6	2	0.18 ± 0.15
	30	10	0.17 ± 0.02
	90	30	0.17 ± 0.01
Paired	6	2	0.47 ± 0.07
	30	10	0.26 ± 0.07
	90	30	0.25 ± 0.01
Global	6	2	0.92 ± 0.04
	30	10	0.88 ± 0.01
	90	30	0.80 ± 0.01

C.4 HOW THE TOY MODEL MAPS TO REAL-WORLD MODELS

A central question is how model scale affects adversarial vulnerability. Empirical evidence suggests that larger models can be somewhat more robust to adversarial attacks, but the effect is weak. Under adversarial training, scaling trends are more pronounced; however, the resulting robustness gains are typically specific to the attack type used during training rather than broadly transferable across attacks (Howe et al., 2025).

Two opposing effects are relevant when models scale. Larger models provide greater representational capacity, but broad tasks such as next-token prediction over internet text also require modelling a long tail of useful concepts. Consequently, increased capacity does not eliminate superposition; indeed, superposition appears to remain prevalent even in frontier models (Lindsey et al., 2025). Dictionary-learning evidence is consistent with this view: larger models require increasingly large SAE dictionaries (Gao et al., 2025), suggesting that the number of represented features grows with model size. Thus, the representational pressure that drives superposition—compressing many features into limited dimensions—does not vanish under scale.

Because both superposition and adversarial vulnerability persist in large-scale models, the proposed mechanism may remain relevant across model scales. A useful direction for future work is to determine how superposition geometry changes with scale and whether those geometric changes can be used to reduce vulnerability.

C.4.1 SEPARATING SUPERPOSITION EFFECTS FROM CAPACITY REDUCTION

To separate the effect of superposition pressure from the effect of capacity reduction, we keep the bottleneck dimension fixed while varying the number of represented classes. Specifically, we fix the bottleneck dimension at $m = 2$ and vary the number of classes k to manipulate superposition pressure. Model accuracy, robust accuracy, and transferability are reported in Tab. 5. These controls indicate that increased superposition pressure (k/m), rather than reduced bottleneck capacity alone, drives the observed adversarial vulnerability.

C.4.2 ADDITIONAL EXAMPLES OF AEXS IN TOY MODEL

Fig. 1 illustrates how adversarial attacks exploit interference between latent features in superposition. Here, we provide additional visual examples (Fig. 5) that instantiate the same mechanism under related configurations. Specifically, we show:

- Fig. 5a provides an additional instance of the setup in Sec. 3.1 ($m = 2, k = 7$) using an ℓ_2 -norm PGD attack, displaying the IPP and latent-space displacement that produce misclassification.
- Fig. 5b shows the same qualitative setup ($m = 2, k = 7$) using an ℓ_∞ -norm PGD attack on a less sparse input.
- Fig. 5c presents an example with increased bottleneck dimensionality ($m = 3, k = 7$) and an accompanying ℓ_2 -norm PGD attack. It also reports the feature-vector similarity matrix used to compute geometric similarity.

D CIFAR-10 EXPERIMENTS

To test whether the mechanisms observed in the toy models extend to a more complex vision setting, Sec. 4 studies CIFAR-10 (Krizhevsky, 2009) models with an engineered bottleneck. This appendix section specifies the experimental setup and reports extended results.

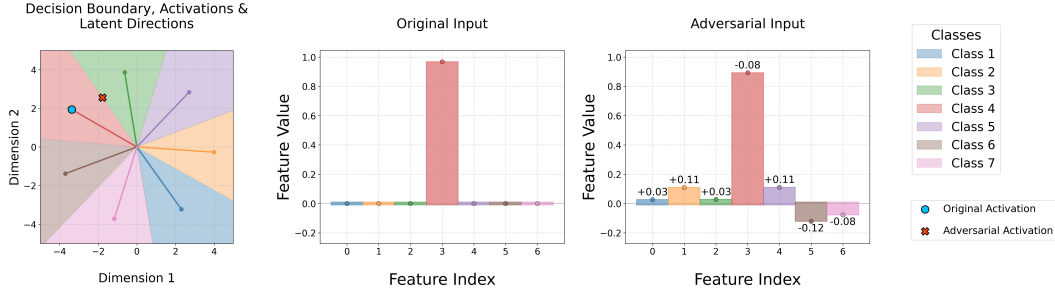
D.1 ARCHITECTURE & TRAINING INFORMATION

The base ViT (Dosovitskiy et al., 2020) architecture uses 6 transformer layers, embedding dimension (d) 512, and 8 attention heads per transformer layer. CIFAR-10 images of size 32×32 pixels are partitioned into 4×4 patches. The Multilayer Perceptron (MLP) in each transformer block has hidden dimension 512, and the model uses learned positional embeddings.

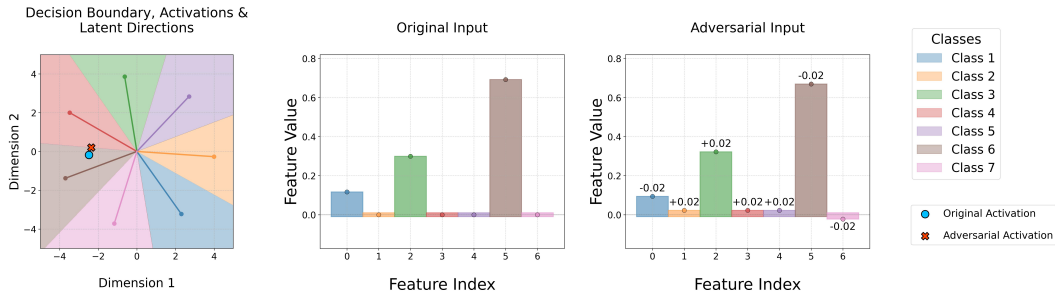
The bottleneck architecture consists of a linear encoder followed by a linear decoder. The encoder projects pre-classification activations from the ViT backbone into an m -dimensional latent space

Table 5: Performance metrics across correlation types, perturbation budgets (ε), and numbers of classes (k).

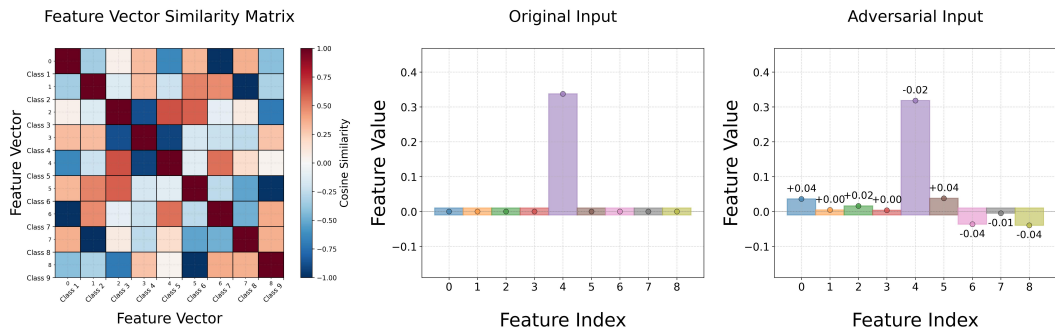
Correlation Type	Parameters		Performance (%)		
	ε	k	Accuracy	Robust Acc.	Attack Trans.
Uncorrelated	0.05	2	100.0	100.0	0.0
		4	100.0	94.9	21.2
		6	99.9	87.0	9.8
		8	99.8	77.7	17.5
	0.1	2	100.0	100.0	0.0
		4	100.0	88.1	9.8
		6	99.7	75.4	9.9
		8	99.6	56.3	23.5
	0.5	2	100.0	100.0	0.0
		4	100.0	99.4	0.0
		6	99.9	62.9	30.5
		8	99.5	24.9	39.0
Fully Correlated	0.05	2	100.0	100.0	0.0
		4	100.0	95.8	98.4
		6	97.3	70.6	98.4
		8	93.0	57.4	97.2
	0.1	2	100.0	100.0	0.0
		4	100.0	88.7	98.5
		6	97.7	45.6	99.7
		8	94.2	34.8	98.6
	0.5	2	100.0	100.0	0.0
		4	100.0	90.0	100.0
		6	97.2	34.6	100.0
		8	93.0	32.0	100.0



(a) An ℓ_2 -norm attack changing the classification of an input of class 4 to class 3. The left plot shows original and adversarial activations in latent space, along with representation directions. The right plots show original and perturbed input feature values.



(b) An ℓ_∞ -norm attack changing the classification of an input of class 6 to class 4. The left plot shows original and adversarial activations in latent space relative to class latent directions. The right plots show original and perturbed input feature values.



(c) An ℓ_2 -norm adversarial attack in a 7-class setup with an increased bottleneck dimension $m = 3$. The leftmost plot now shows the cosine similarity matrix between pairs of latent representations for each of the classes.

Figure 5: Visualisations of AExs in the toy model, supplementing Figure 1 from the main paper by illustrating attack mechanisms in activation space and input space under varied conditions.

with $m \in \{2, 3, 5, 10\}$. The decoder maps these m -dimensional representations back to the $k = 10$ CIFAR-10 class logits.

The base ViT is trained on CIFAR-10 for 250 epochs using CE loss, Adam with default PyTorch parameters, learning rate 0.001, batch size 512, dropout, and a cosine-annealing learning-rate scheduler. Training is repeated across five random seeds to measure variability. After base-model training, the ViT parameters are frozen and the bottleneck layer is trained for 30 epochs with learning rate 0.001. Preprocessing applies RandomCrop with 4-pixel padding, resizing to the target size (32x32 by default), and RandomHorizontalFlip augmentation.

D.1.1 NORMALISED ROBUST ACCURACY ACROSS PERTURBATION MAGNITUDES

Fig. 4 (right) reports normalised robust accuracy as a function of bottleneck dimension for one value of ϵ . This subsection extends that analysis by reporting normalised robust accuracy under PGD attacks with multiple perturbation magnitudes. Results are given for both ℓ_2 -norm (Tab. 7) and ℓ_∞ -norm (Tab. 6) PGD attacks, providing a more complete characterisation of how superposition pressure and attack strength jointly affect robustness.

For these evaluations, AExs are generated using PGD with 100 iterations and step size (α) 0.01. Robust accuracy is evaluated on 500 samples for each configuration, and we report mean normalised robust accuracy with standard deviation across five random seeds.

Table 6: Mean normalised robust accuracy ($\pm$ standard deviation across 5 seeds) for ViT models with different bottleneck dimensions (m) on CIFAR-10, subjected to ℓ_∞ -norm PGD attacks of varying perturbation magnitudes (ϵ). Robust accuracy is normalised by the clean accuracy of each bottlenecked model. These results support the findings of Sec. 4, demonstrating a similar trend across ϵ .

ϵ	Bottleneck Dimension (m)			
	2	3	5	10
0.001	96.0% $\pm$ 0.4%	97.4% $\pm$ 0.7%	97.9% $\pm$ 0.4%	98.1% $\pm$ 0.3%
0.01	61.7% $\pm$ 3.6%	69.6% $\pm$ 3.5%	77.0% $\pm$ 0.9%	81.8% $\pm$ 3.2%
0.05	4.9% $\pm$ 1.5%	6.7% $\pm$ 1.6%	9.3% $\pm$ 0.6%	10.5% $\pm$ 0.8%
0.1	0.1% $\pm$ 0.2%	0.2% $\pm$ 0.3%	0.2% $\pm$ 0.4%	0.2% $\pm$ 0.4%
0.5	0.0% $\pm$ 0.0%	0.0% $\pm$ 0.0%	0.0% $\pm$ 0.0%	0.0% $\pm$ 0.0%

Table 7: Mean normalised robust accuracy ($\pm$ standard deviation across 5 seeds) for ViT models with different bottleneck dimensions (m) on CIFAR-10, subjected to ℓ_2 -norm PGD attacks of varying perturbation magnitudes (ϵ). Robust accuracy is normalised by the clean accuracy of each bottlenecked model. These results support the findings in Section 4 of the main paper, showing decreasing robustness with smaller m (increased superposition) and larger ϵ .

ϵ	Bottleneck Dimension (m)			
	2	3	5	10
0.1	90.4% $\pm$ 1.7%	91.8% $\pm$ 0.7%	94.6% $\pm$ 1.2%	95.0% $\pm$ 0.7%
0.5	58.5% $\pm$ 5.0%	60.8% $\pm$ 5.0%	69.2% $\pm$ 1.8%	72.6% $\pm$ 2.8%
1.0	41.7% $\pm$ 4.8%	44.0% $\pm$ 3.0%	50.4% $\pm$ 0.6%	54.9% $\pm$ 1.7%
2.0	34.0% $\pm$ 4.6%	36.2% $\pm$ 3.3%	41.5% $\pm$ 1.3%	47.4% $\pm$ 2.4%
5.0	30.2% $\pm$ 4.1%	32.9% $\pm$ 3.5%	37.3% $\pm$ 1.9%	42.7% $\pm$ 1.0%

D.1.2 ATTACK TRANSFERABILITY ACROSS PERTURBATION MAGNITUDES

We report attack-transferability measurements across perturbation magnitudes (ϵ) and bottleneck dimensions (m). Tab. 9 gives ℓ_2 -norm transferability, and Tab. 8 gives ℓ_∞ -norm transferability.

D.2 CORRELATIONS IN CLASS FEATURES

In Sec. 3, correlations between input features determine superposition arrangements. In the CIFAR-10 setting, the relevant correlations are not correlations between raw input classes, but correlations among the representations at the pre-classification layer. At the classification layer, these correlations are plausibly related to the confusion matrix, *i.e.* the pattern by which each class is misclassified as other classes. Classes that cluster together are therefore those that the network represents as similar and tends to confuse. To test this interpretation, we repeat the experiment by fine-tuning a base ViT using timm/vit_base_patch16_384, which was trained on ImageNet-21k (14 million images, 21,843 classes) and ImageNet (1 million images, 1,000 classes). After fine-tuning on CIFAR-10 and applying the same bottleneck training procedure as in Sec. 4, the resulting geometry shows random ordering across initialisations with approximately equal spacing between features (*i.e.* neuron collapse Kothapalli (2023)). In this case, performance is near 100%, so the confusion matrix is close to the identity and the superposed geometry loses its structured clustering. By contrast, models trained only on CIFAR-10 converge to similar geometries because they share the same learned difficulty structure: the same class pairs are difficult to distinguish, producing consistent superposition patterns.

Table 8: Attack transferability (%) for ℓ_∞ -norm PGD attacks on CIFAR-10 ViT models. Each row reports transfer from a source model trained with the indicated seed (for example, Seed 10) to three target models trained with the seeds listed in the corresponding sub-header (for example, Seeds 20/30/40). Slash-separated entries give transferability to those three target seeds in order. All models in a row share bottleneck dimension m . The ‘Mean $\pm$ Std’ column averages transferability over all 12 source-target seed pairings for each (ϵ , m) configuration. These results support the claim in Section 4 that stronger superposition can yield more consistent latent geometry and higher transferability.

ϵ	m	Seed 10 vs. Seeds 20/30/40	Seed 20 vs. Seeds 10/30/40	Seed 30 vs. Seeds 10/20/40	Seed 40 vs. Seeds 10/20/30	Mean $\pm$ Std
0.001	2	77.8/66.7/44.4	25.0/62.5/25.0	72.7/72.7/72.7	58.3/75.0/58.3	59.3 $\pm$ 17.7
	3	57.1/28.6/28.6	14.3/28.6/28.6	63.6/72.7/72.7	42.9/42.9/42.9	43.6 $\pm$ 18.4
	5	70.0/50.0/50.0	57.1/71.4/57.1	50.0/25.0/25.0	16.7/50.0/33.3	46.3 $\pm$ 16.9
	10	55.6/55.6/66.7	12.5/37.5/25.0	16.7/33.3/16.7	28.6/71.4/28.6	37.3 $\pm$ 19.3
0.01	2	60.3/52.6/48.7	63.8/42.0/60.9	53.8/40.9/52.7	51.2/55.8/46.5	52.4 $\pm$ 6.9
	3	42.9/46.4/50.0	46.4/42.9/49.1	44.2/38.9/44.2	42.9/45.1/41.8	44.6 $\pm$ 3.0
	5	43.3/41.1/43.3	39.2/30.4/41.8	46.0/40.2/47.1	41.2/35.3/32.9	40.2 $\pm$ 4.8
	10	42.5/46.0/44.8	38.0/39.4/46.5	43.5/47.8/47.8	32.6/40.0/35.8	42.1 $\pm$ 4.7
0.05	2	35.0/36.4/37.8	38.8/36.0/46.3	37.4/31.3/41.7	42.0/42.0/40.7	38.8 $\pm$ 3.8
	3	29.9/29.9/37.1	30.4/30.1/35.6	30.2/30.2/35.1	30.1/31.9/34.4	32.1 $\pm$ 2.6
	5	26.8/25.3/29.2	25.2/21.5/26.4	28.0/28.0/26.5	26.0/26.6/21.9	25.9 $\pm$ 2.2
	10	27.1/24.9/30.4	21.6/23.0/28.4	25.3/28.1/30.6	21.0/24.9/20.1	25.4 $\pm$ 3.4
0.1	2	34.2/35.6/38.2	36.7/34.9/44.5	37.9/30.0/40.8	41.2/42.0/40.3	38.0 $\pm$ 3.8
	3	29.6/28.9/35.2	28.6/29.2/34.8	29.1/28.2/34.0	28.8/31.1/32.8	30.9 $\pm$ 2.5
	5	25.4/23.3/27.0	23.8/19.6/24.6	25.9/26.7/25.4	24.6/24.3/20.7	24.3 $\pm$ 2.1
	10	24.8/22.6/29.7	20.1/21.6/26.1	22.5/25.5/28.0	19.5/23.3/18.7	23.5 $\pm$ 3.3

D.2.1 RESNET-18

To address concerns on architectural generalisation, we conduct the same experiments using a ResNet-18 (He et al., 2016) architecture as the base model as opposed to a ViT. We achieve a slightly higher

Table 9: Attack transferability (%) for ℓ_2 -norm PGD attacks on CIFAR-10 ViT models. The source-to-target transfer format matches Tab. 8. These ℓ_2 results further support the claim in Sec. 4 that stronger superposition increases attack transferability.

ϵ	m	Seed 10 vs. Seeds 20/30/40	Seed 20 vs. Seeds 10/30/40	Seed 30 vs. Seeds 10/20/40	Seed 40 vs. Seeds 10/20/30	Mean $\pm$ Std
0.1	2	70.6/64.7/41.2	60.0/50.0/60.0	64.3/60.7/57.1	48.1/63.0/51.9	57.6 ± 8.0
	3	39.1/39.1/47.8	50.0/60.7/50.0	48.1/55.6/51.9	48.0/28.0/40.0	46.5 ± 8.4
	5	43.5/34.8/43.5	31.6/31.6/42.1	42.9/35.7/21.4	30.4/39.1/34.8	35.9 ± 6.4
	10	45.5/59.1/45.5	19.0/28.6/28.6	37.5/43.8/43.8	23.8/52.4/38.1	38.8 ± 11.4
0.5	2	59.6/52.1/48.9	60.8/35.4/60.8	54.1/39.4/51.4	43.8/49.5/46.7	50.2 ± 7.7
	3	38.7/40.6/48.1	36.1/36.8/41.7	37.4/36.5/43.5	34.4/34.4/36.7	38.7 ± 3.9
	5	38.4/32.0/37.6	29.4/25.7/32.1	39.1/33.9/38.3	31.8/29.9/32.7	33.4 ± 4.0
	10	38.3/37.4/37.4	26.5/28.4/31.4	32.4/35.3/35.3	23.4/28.2/25.0	31.6 ± 5.0
1.0	2	50.8/43.7/44.4	51.7/36.4/50.0	46.1/32.9/44.7	42.7/46.7/45.3	44.6 ± 5.3
	3	35.7/36.9/41.7	31.8/36.4/42.6	33.7/32.0/37.3	34.9/32.0/35.5	35.9 ± 3.3
	5	32.1/28.9/33.2	30.9/23.0/30.9	35.9/31.5/33.7	27.7/28.2/27.1	30.3 ± 3.3
	10	32.3/32.3/38.5	23.7/27.7/29.9	29.1/32.0/34.3	22.2/25.6/21.1	29.1 ± 5.0
2.0	2	47.0/41.6/44.3	48.2/36.2/48.2	43.9/33.5/42.1	43.4/43.4/42.8	42.9 ± 4.2
	3	31.9/37.7/40.3	31.1/35.6/40.6	32.6/31.1/36.8	35.4/33.3/34.3	35.1 ± 3.2
	5	29.9/25.4/32.1	26.3/22.1/26.7	32.3/31.8/29.5	27.9/27.9/23.6	28.0 ± 3.2
	10	30.8/30.4/35.7	22.5/24.9/28.6	24.8/29.7/32.7	20.4/26.1/19.0	27.1 ± 4.9
5.0	2	44.7/46.7/40.8	45.3/37.3/48.7	42.7/33.9/41.5	42.3/42.3/40.6	42.2 ± 3.8
	3	32.3/35.8/40.8	31.3/32.6/39.9	31.0/30.0/33.8	35.0/33.0/33.0	34.1 ± 3.2
	5	30.6/24.1/28.6	28.3/20.9/24.8	31.7/31.3/27.4	28.1/29.0/23.2	27.3 ± 3.3
	10	29.8/27.4/32.7	19.7/24.1/28.9	24.9/29.9/31.2	18.8/25.8/19.2	26.0 ± 4.6

92% clean accuracy (compared to 89% for ViT). We observe the robust accuracy falls slightly faster for ResNet-18 than the ViT at the same ϵ values. Nevertheless, we observe similar declining trends in normalised robust accuracy and increasing transferability as bottleneck pressure increases.

E MODULAR ADDITION

Sec. 5 introduces a modular-addition task for analysing adversarial vulnerability when the relevant features are orthogonal. This section gives the detailed experimental setup, additional results, and further discussion of how this representational analysis informs robustness.

E.1 LEARNED ALGORITHMIC REPRESENTATION IN MODULAR ADDITION

As detailed in Nanda et al. (2023) and Prieto et al. (2025b), the network learns a trigonometric algorithm for the task: the encoder projects inputs onto non-basis-aligned linear representations encoding $\sin(\omega_k a)$ and $\cos(\omega_k a)$ at key frequencies $\omega_k = 2\pi k/P$. These features are represented orthogonally, *i.e.* not in superposition, and the sinusoidal values are linearly probeable. The subsequent MLP applies sum-angle identities to compute $\cos((a+b)\omega_k)$ for multiple k , and these components constructively interfere at the logits to select the correct class. AExs are generated by relaxing the one-hot constraint and perturbing input $\mathbf{x}_j \in \mathbb{R}^P$ (encoding integer j) to $\mathbf{x}' = \mathbf{x} + \delta$ using ℓ_2 - and ℓ_∞ -norm PGD. For simplicity, only a is attacked while b is held fixed.

Following Nanda et al. (2023), the learned algorithm has explicit trigonometric structure. W_e encodes sine and cosine values of integers at the key frequencies, and these quantities are linearly decodable.

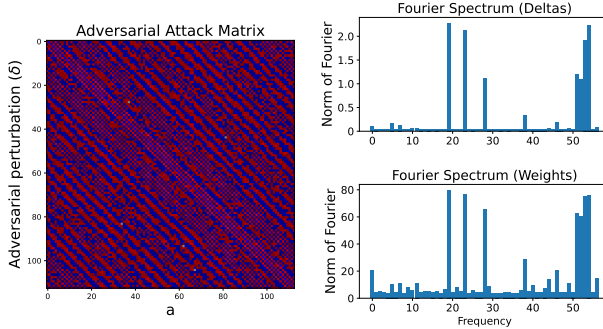


Figure 6: Left: each column gives the mean attack δ added to a to successfully attack $a + b \pmod p$. Right: norms of the Fourier components in δ and $W_{MLP_{out}}$.

Table 10: Minimum ℓ_2 -norm budget required to successfully attack $0 + 0 \pmod p$ for each attack type.

Attack Method	Success ϵ
PGD	0.17
Informed Attack	0.22
Single a	0.97
ℓ_∞ Noise	2.04
Uniform δ	6.20

Scaling the corresponding feature vector changes the encoded sinusoidal value itself rather than only changing activation intensity. Integer embeddings $\mathbf{W}_e^j$ project onto a Fourier-like basis, inducing circular or periodic structure in embedding space. Unlike the geometries produced by data correlations in the toy model, this circularity arises from sine/cosine encoding of non-correlated input integers.

The model uses approximately seven key frequencies (ω_k). Each embedding W_e^j for input integer x_j primarily encodes fourteen associated trigonometric features, $\cos(\omega_i x_j)$ and $\sin(\omega_i x_j)$ for $i \in \{1, 2, \dots, 7\}$. The concatenated 200-dim. vector therefore supplies approximately 28 key trigonometric features to the MLP, fourteen for a and fourteen for b . The MLP uses these features through sum-angle identities (e.g., $\cos(A+B) = \cos A \cos B - \sin A \sin B$) to compute $(x_1 + x_2) \pmod P$. The algorithmic brittleness analysed in Section 5 arises because adversarial perturbations corrupt these learned trigonometric features.

E.2 EXPERIMENT DETAILS FOR MODULAR ADDITION TASK

The network is trained on addition modulo 113. Inputs are integer pairs (a, b) with $a, b \in \{0, \dots, 112\}$. Each integer is mapped to a 100-dimensional vector using a shared embedding matrix $\mathbf{W}_E \in \mathbb{R}^{113 \times 100}$. The embeddings $\mathbf{W}_E^a$ and $\mathbf{W}_E^b$ are concatenated into a 200-dim. feature vector, which is passed to a 3-hidden-layer MLP with 200 ReLU neurons per layer. The output layer produces 113 classification logits. The model is trained end-to-end with cross-entropy loss and AdamW (weight decay 4), reaching 100% test accuracy. AExs are generated with PGD under ℓ_2 and ℓ_∞ norms across varying ϵ . Attacks perturb the embedding $\mathbf{W}_E^a$ while holding integer b fixed, adding perturbation δ to induce misclassification of $(a + b) \pmod{113}$.

E.3 ALGORITHMIC BRITTLINESS

Frequency-aligned perturbations induce misclassification by altering the sinusoidal quantities read from the encoder features. For intuition, consider the hypothesised neuron-aligned case: $\mathbf{W}_e$ contains a row corresponding to the sine or cosine component for each ω_k , and the element at column $\mathbf{w}_e^a$ contains $\sin(\omega_k a)$ or $\cos(\omega_k a)$. For input a , the one-hot vector $\mathbf{x}_a$ selects $\mathbf{w}_e^a$, yielding $\mathbf{h}_a = \mathbf{w}_e^a$. The perturbation δ transforms the one-hot vector $\mathbf{x}_a$ into a dense vector $\mathbf{x}'_a$, so $\mathbf{h}'_a = \mathbf{W}_e \mathbf{x}'_a = (1 + \epsilon_a) \mathbf{h}_a + \sum_{k \neq a} \epsilon_k \mathbf{w}_e^k$. The additional contributions from $k \neq a$ sinusoidal terms corrupt the values supplied to the MLP, disrupting its downstream computation and producing misclassification. Because the model’s algorithm uses constructive contributions from each ω_k at the logits, disrupting only one ω_k is insufficient; the remaining components still strongly support the correct answer. Effective attacks must therefore identify δ that changes multiple frequency components simultaneously.

E.4 ANALYTIC CONSTRUCTION OF ATTACKS AND BASELINE COMPARISONS

The main paper shows that understanding the learned trigonometric algorithm in the modular-addition task enables analytic construction of adversarial attacks. Sec. 5 benchmarks this ‘informed attack’ against PGD and naive baselines. This section specifies the baseline attack methods:

- **PGD Attack:** Standard PGD used to optimise an adversarial perturbation.
- **Informed Attack:** The analytically constructed perturbation formed from combinations of cosines.
- **Single a :** The minimum perturbation to a single input coordinate required to induce misclassification.
- **ℓ_∞ Noise:** Random noise δ_{noise} is added to input a , with each component sampled independently and uniformly from $[-\epsilon, \epsilon]$. We report the smallest ϵ that induces misclassification.
- **Uniform δ :** The perturbation vector δ is constrained to have equal components, i.e., $\delta_j = c \cdot \mathbf{1}$ for scalar c . We report the magnitude of c required for δ to cause misclassification.

E.5 DETAILS ON CERTIFIED ROBUST TRAINING

To evaluate the effect of defences on the identified vulnerability, we perform certified robust training intended to make the network provably robust to ℓ_∞ -norm perturbations. The certified training method is an abstraction-based technique (Wong & Kolter, 2018; Raghunathan et al., 2018; Palma et al., 2022), specifically RSIP-IBP (Zhang et al., 2020; Henriksen & Lomuscio, 2023). RSIP-IBP uses Interval Bound Propagation (IBP) (Katz et al., 2017; Ehlers, 2017) and Reversed Symbolic Interval Propagation (RSIP) to compute reachable output sets for network layers, enabling formal robustness certification within a specified perturbation budget. The training loss combines a standard loss term with an RSIP loss term. Certified training is run for 1000 epochs, with the first 200 epochs used as warm-up. The learning rate is 1×10^{-3} and weight decay is 1×10^{-4} . We use the same train/test splits as in standard model training. Tab. 11 summarises certified-training results across training perturbation budgets (ϵ_{train}) and evaluation attack budgets (ϵ_{attack}).

Table 11: Verified robust accuracy (%) for modular-addition models under ℓ_∞ -norm perturbations. The table compares a standard pre-robustification model with models certifiably trained using RSIP-IBP under different training perturbation budgets (ϵ_{train}). Accuracies are evaluated at multiple attack budgets (ϵ_{attack}). The results support the discussion in Section 5.1 of the main paper: certified training improves robustness at specified ϵ_{train} , but attacks at larger ϵ_{attack} still exploit the same algorithmic sensitivities.

Attack Epsilon (ϵ_{attack})	Train Epsilon (ϵ_{train})	Standard Acc. (Pre-Rob., %)	Robust Acc. (Pre-Rob., %)	Standard Acc. (Robust, %)	Robust Acc. (Robust, %)
5.0×10^{-4}	10^{-4}	100.0	100.0	100.0	100.0
	10^{-3}				100.0
1.0×10^{-3}	10^{-4}	100.0	76.0	100.0	93.8
	10^{-3}				100.0
2.0×10^{-3}	10^{-4}	100.0	0.0	100.0	0.0
	10^{-3}				95.7
5.0×10^{-3}	10^{-4}	100.0	0.0	100.0	0.0
	10^{-3}				85.3

As discussed in Sec. 5, subsequent PGD attacks on the robustly trained models retain the same qualitative structure: successful perturbations continue to target the key frequency components used by the learned algorithm. The main effect of certified training is to increase the perturbation magnitude required for misclassification at certified epsilons, rather than to change the alignment between successful attack vectors and the model’s algorithmic sensitivities.

E.6 ON A POTENTIAL RECONCILIATION WITH SUPERPOSITION VULNERABILITY

Although the vulnerability analysed here occurs without feature superposition, it resembles superposition-driven vulnerabilities in one respect: both depend on sparsity assumptions in the input representation. This motivates our *reconciliation conjecture*: algorithmic vulnerabilities can persist in networks with orthogonal representations when upstream processing violates sparsity assumptions. If earlier layers produce noisy or non-sparse features, potentially through adversarial interference, these deviations can propagate into otherwise well-isolated downstream circuits. A

circuit that behaves correctly on clean inputs may then fail because its received input is effectively already perturbed. This provides a plausible route by which algorithm-disruption mechanisms, such as the frequency attacks studied here, could be activated even without local superposition. More generally, the interaction between upstream superposition, downstream algorithmic brittleness, and layer-specific adversarial mechanisms remains underexplored. We leave detailed empirical and theoretical investigation of this interaction to future work.

F INSPECTING ADVERSARIAL PERTURBATIONS VIA SAE FEATURES

The main-paper analyses focus on settings with either explicitly defined class features (the toy model) or class-level bottleneck representations (the ViT experiments). In large practical networks, however, interference may occur among unlabelled latent features across many layers. As discussed in the future-work section, SAEs provide an unsupervised method for extracting more linear features from NNs (Bricken et al., 2023). This appendix presents a preliminary investigation of whether SAE-extracted features can characterise how a large-scale ViT internally processes AExs relative to clean inputs. These results are exploratory and are intended to assess feasibility for future feature-level analyses of adversarial phenomena in large models, rather than to establish a conclusive additional finding. We first specify the experimental setup and validate SAE behaviour on adversarial inputs, then present layer-wise feature-difference analyses.

F.1 EXPERIMENT SETUP

We analyse SAE-extracted features from the vision encoder of LAION/CLIP-ViT-B-32-DataComp.XL-s13B-b90K, a 12-layer ViT with $\approx 70\%$ zero-shot accuracy on ImageNet-1k Deng et al. (2009). We use open-source Prisma SAEs that map the 768-dimensional activation space to a dictionary of 49,152 features ($64\times$ expansion) (Joseph et al., 2025). Across all layers, we examine SAEs trained on either mean patch-token representations or CLS-token representations. We analyse features above a high activation threshold (0.1), which captures changes among strongly activated features, and a low threshold (0.001), which includes weaker activations. For feature comparisons, we generate 10,000 original/successfully attacked image pairs using ℓ_∞ -norm attacks with $\epsilon = 0.02$. This setup provides an initial test of how adversarial perturbations appear in the SAE feature basis.

F.2 SAE VALIDITY FOR ADVERSARIAL INPUTS

Because AExs are out-of-distribution inputs and the SAEs are trained under standard conditions, we first perform a basic validation. This subsection measures the effect on model accuracy and SAE reconstruction when SAEs are inserted at inference time for clean and adversarial inputs. The goal is to verify that the SAEs process classification-relevant features even under adversarial conditions before interpreting differences in feature activations.

F.3 LAYER-WISE SAE FEATURE DIFFERENCES

This subsection presents exploratory visualisations comparing SAE feature activations across ViT layers for original, ℓ_∞ adversarial, and random noisy inputs. The visualisations identify candidate measurements for future analysis of how adversarial attacks alter sparse feature representations. We examine:

- Activation count: Histograms of the number of SAE features activated above a fixed threshold in a layer for original, adversarial, and noisy images; this measures whether attacks alter feature sparsity.
- Feature overlap: The number of shared activated features between original-versus-adversarial and original-versus-noisy conditions; this measures whether attacks activate a distinct feature set.
- Jaccard Similarity: The Jaccard similarity between sets of activated features, providing a normalised overlap metric.
- Mean activation: Histograms of mean activation values for features active above threshold; this measures changes in feature intensity.
- Distinct Features: The number of unique features activated across an image set for each condition.

Table 12: Effect on ImageNet-1k classification accuracy when a pre-trained SAE is inserted one layer at a time during inference for original inputs, ℓ_∞ adversarial inputs ($\epsilon = 0.02$), and random noisy inputs. ‘Accuracy’ denotes ViT top-1 accuracy. ‘Original input + SAE’ indicates that clean ViT activations at the specified layer are passed through the SAE, after which the reconstruction is passed to the next layer. The slight ‘de-attacking’ effect, measured by improved accuracy for ‘Adv. input + SAE’ relative to ‘Adv. input’, suggests that SAEs may filter some adversarial noise; however, the model still largely misclassifies, indicating that salient adversarial changes remain processed.

Metric	Layer 2	Layer 5	Layer 8	Layer 11
Accuracy				
Original input	1.00	1.00	1.00	1.00
Original input + SAE	0.99	0.99	0.99	0.86
Adv. input	0.00	0.00	0.00	0.00
Adv. input + SAE	0.02	0.01	0.01	0.11
Noisy input	0.98	0.98	0.98	0.98
Noisy input + SAE	0.98	0.98	0.98	0.85
SAE Improvement				
Original input	-0.01	-0.01	-0.01	-0.14
Adv. input	0.02	0.01	0.01	0.11
Noisy input	-0.00	-0.00	-0.00	-0.13

F.4 CLS TOKEN SAE FEATURES WITH HIGH ACTIVATION THRESHOLD (0.1)

This subsection analyses SAE features derived from CLS tokens using only features activated above threshold 0.1. Figures 7 through 10 compare activation-count histograms, feature overlap, Jaccard similarity, and mean-activation histograms for original, adversarial, and random noisy inputs across ViT layers.

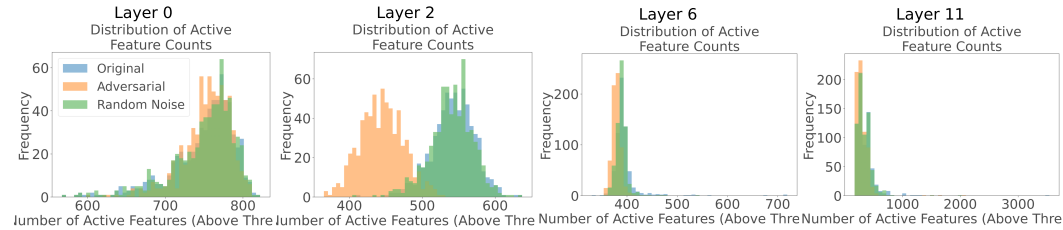


Figure 7: Histogram of activation counts for CLS token SAE features (threshold 0.1).

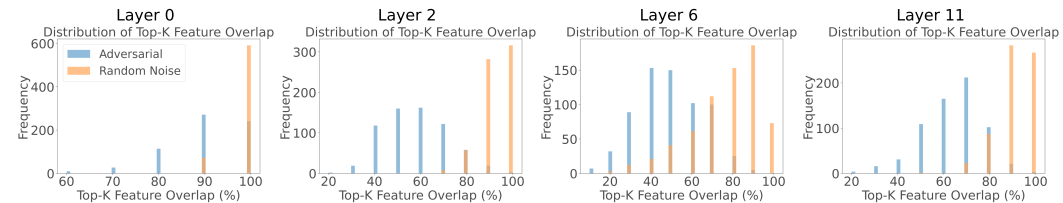


Figure 8: Feature overlap between original vs. attacked and original vs. noisy images for CLS token SAE features (threshold 0.1).

F.5 CLS TOKEN SAE FEATURES WITH LOW ACTIVATION THRESHOLD (0.001)

We next examine CLS-token SAE features using a lower activation threshold of 0.001, which includes weaker activations in addition to the most strongly activated features. The following figures report

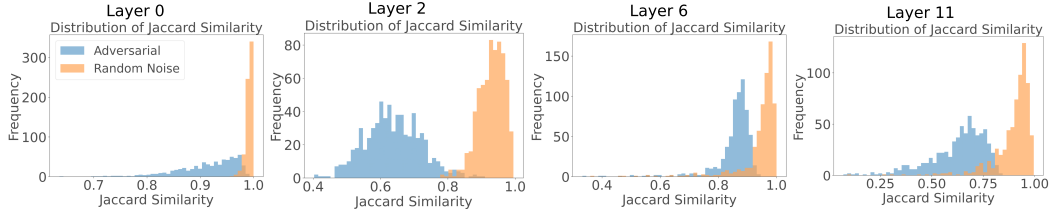


Figure 9: Comparison of Jaccard similarity for CLS token SAE features (threshold 0.1).

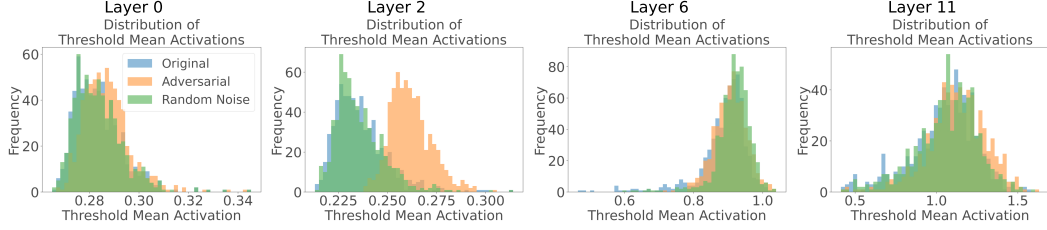


Figure 10: Histogram of mean thresholds for CLS token SAE features (threshold 0.1).

activation-count histograms (Fig. 11), feature overlap (Fig. 12), Jaccard similarity (Fig. 13), and mean-activation histograms (Fig. 14) across layers for original, adversarial, and noisy inputs.

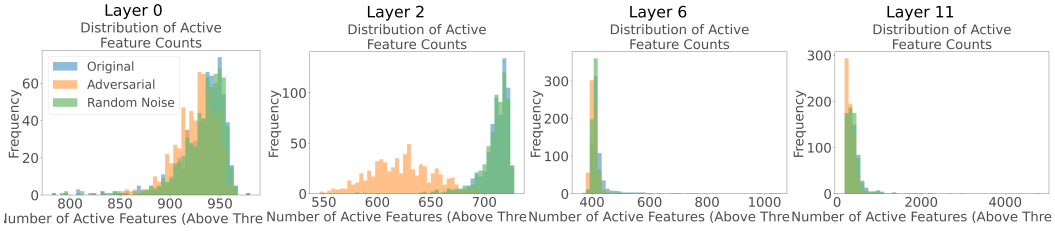


Figure 11: Histogram of activation counts for CLS token SAE features (threshold 0.001), with noise.

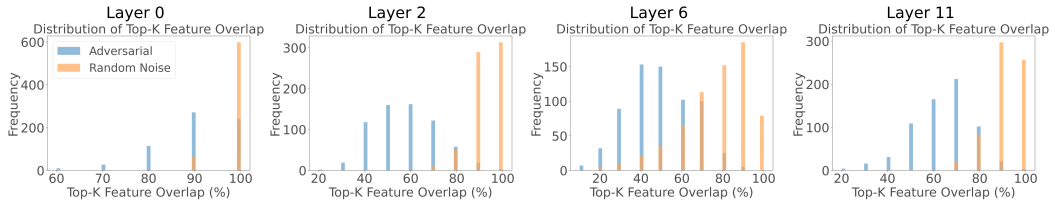


Figure 12: Feature overlap between original vs. attacked and original vs. noisy images for CLS token SAE features (threshold 0.001).

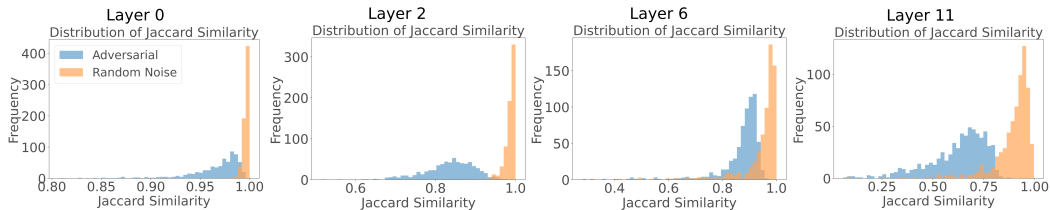


Figure 13: Comparison of Jaccard similarity for CLS token SAE features (threshold 0.001).

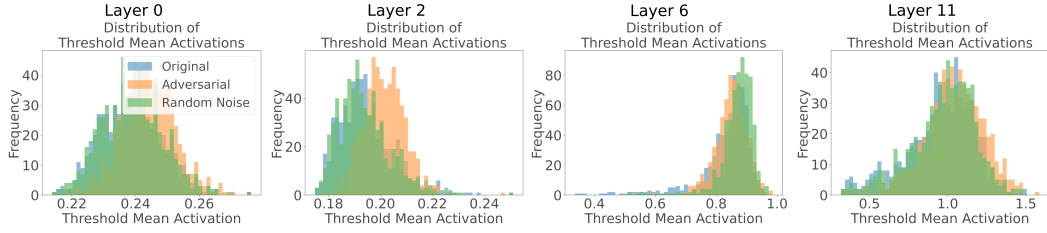


Figure 14: Histogram of mean activations for CLS token SAE features (threshold 0.001), with noise.

F.6 PATCH TOKEN SAE FEATURES WITH HIGH ACTIVATION THRESHOLD (0.1)

This subsection analyses SAE features derived from mean patch-token representations using the high activation threshold 0.1. We compare activation-count histograms (Fig. 15), distinct-feature-count histograms (Fig. 16), feature overlap (Fig. 17), Jaccard similarity (Fig. 18), and mean-activation histograms (Fig. 19) for original, adversarial, and noisy inputs.

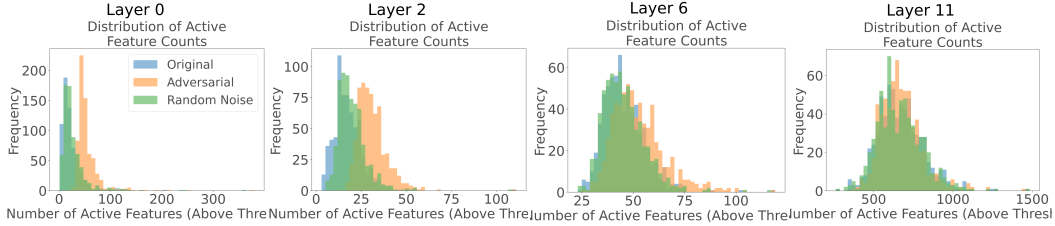


Figure 15: Histogram of activation counts for patch token SAE features (threshold 0.1), with noise.

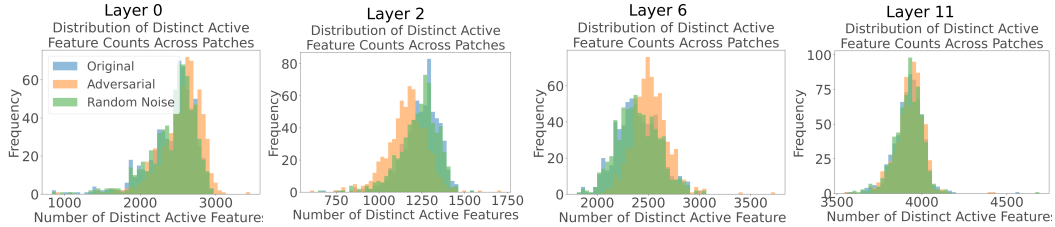


Figure 16: Histogram of distinct patch features for patch token SAE features (threshold 0.1), with noise.

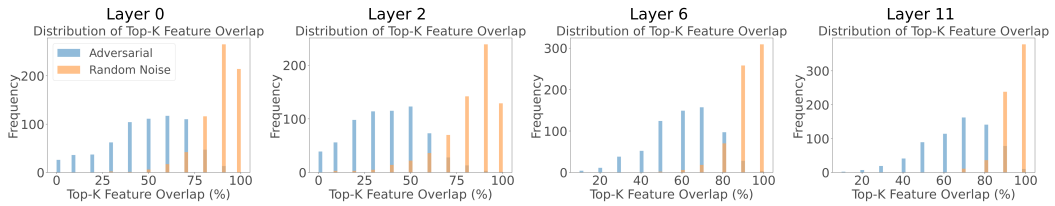


Figure 17: Feature overlap between original vs. attacked and original vs. noisy images for patch token SAE features (threshold 0.1).

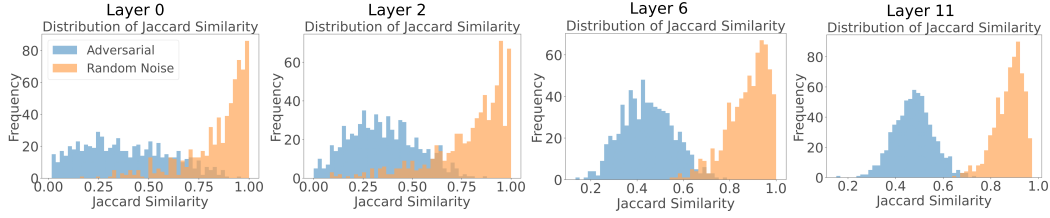


Figure 18: Comparison of Jaccard similarity for patch token SAE features (threshold 0.1).

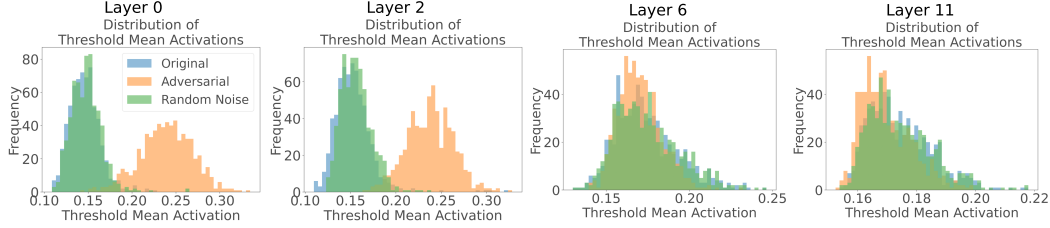


Figure 19: Histogram of mean thresholds for patch token SAE features (threshold 0.1), with noise.

F.7 PATCH TOKEN SAE FEATURES WITH LOW ACTIVATION THRESHOLD (0.001)

Finally, this subsection analyses SAE features from mean patch-token representations using the low activation threshold 0.001. The figures report activation-count histograms (Fig. 20), distinct-feature counts (Fig. 21), feature overlap (Fig. 22), Jaccard similarity (Fig. 23), and mean-activation histograms (Fig. 24) for original, adversarial, and noisy inputs.

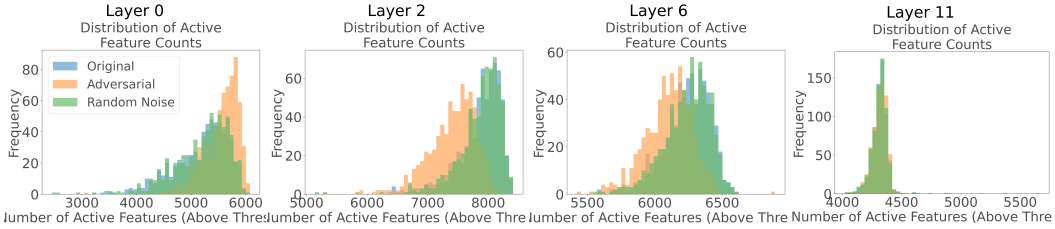


Figure 20: Histogram of activation counts for patch token SAE features (threshold 0.001), with noise.

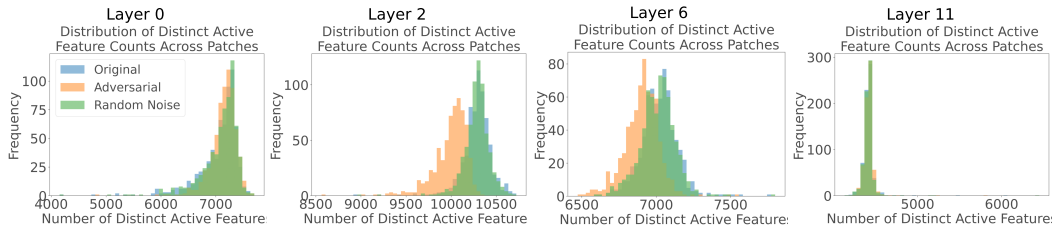


Figure 21: Histogram of distinct patch features for patch token SAE features (threshold 0.001), with noise.

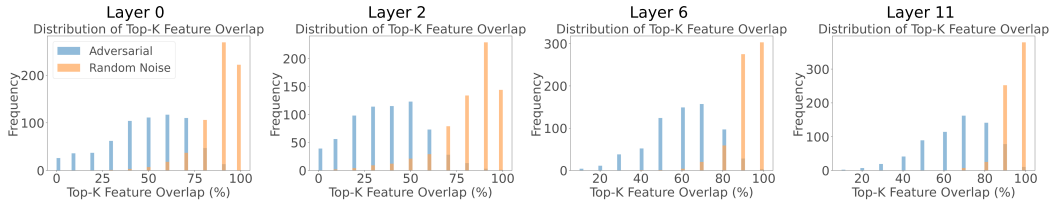


Figure 22: Feature overlap between original vs. attacked and original vs. noisy images for patch token SAE features (threshold 0.001).

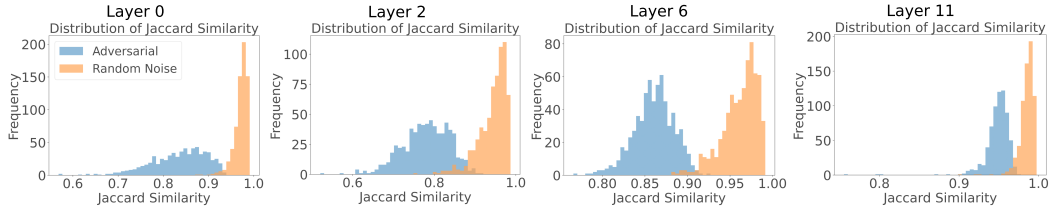


Figure 23: Comparison of Jaccard similarity for patch token SAE features (threshold 0.001).

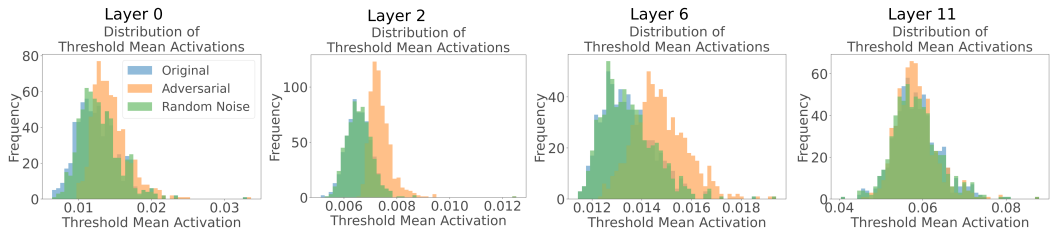


Figure 24: Histogram of mean activations for patch token SAE features (threshold 0.001), with noise.

G ACRONYMS

AEx adversarial example

CE cross entropy

LRH linear representation hypothesis

MLP Multilayer Perceptron

MSE mean squared error

NN neural network

PGD projected gradient descent

SAE sparse autoencoder

VIT vision transformer